Kubernetes Engine

29 March 2022 04:34 PM

Kubernetes Engine: Kubernetes is a container orchestration platform that automates deployment, scaling, networking, and management of containerized applications.

It make easy to orchestrate many containers on many hosts.

- Developed by Google
- It Contains pods, pods contains containers.
- Containers are light weight packages as it has no OS image in it.

- **Cluster:** A Kubernetes cluster is a set of machines (nodes) that work together to run your applications. It consists of at least one master node and multiple worker nodes.
- **Node pool :** It is a group of nodes within a cluster that all have the same configuration.
  - Such as the same machine type and same labels.
- **Nodes:** .
  - It is a group of pods and other resources(Load balancers, persistent disks, cloud operations)
  - A node is a single machine (physical or virtual) that runs your applications as part of a cluster.
- **Pods :** A collection of containers is called as pods. Every pod has IP address.
  - Kubectl get pods (we can get running pods)

- **Containers:** They are light weight ready to use packages in which it contains APPs runtime env and their respective bins/Libs.
- **Volumes:** It stores the data of the pods running in the cluster.
- Configmap and Secrets are down below !!
- **Deployments:** A Kubernetes **Deployment** is a resource that functions similarly to a Google Cloud **MIG (Managed Instance Group)**. While a MIG manages virtual machines, a Deployment manages containers (Pods)—providing automated scaling, rolling updates, and self-healing features."
  - For stateless applications
- **StatefulSets:** A resource used to replica stateful apps and databases.

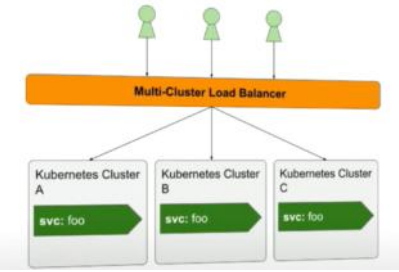
- **Service:**

  - A Kubernetes Service is a static gateway sitting in front of a logical set of ephemeral Pods.
  - Because Pods are constantly created and destroyed (with changing IP addresses), the Service provides a permanent IP address and DNS name.

- **Ingress:** An Ingress is an API object that manages external access to the services in a cluster. It sits in front of your Services and acts as a reverse proxy, traffic router, and entry point.

- Workflow:
  - **External Request (HTTP) ——> Ingress ——> Service (ClusterIP) ——> Pods**

Multi Cluster Ingress :



Feature:

- High **availability** or no downtime
- Disaster **recovery** like backup and restore
- **Scalability** or high performance.

Difference between Service & Ingress

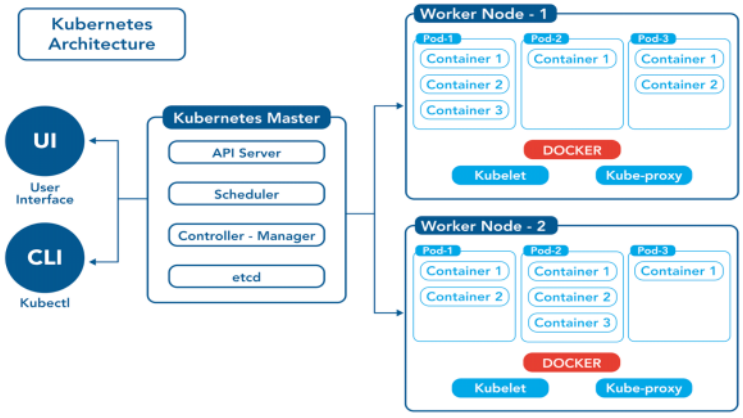
Feature	Service (specifically Type: LoadBalancer)	Ingress
Network Layer	Layer 4 (TCP/UDP, IP + Port)	Layer 7 (HTTP/HTTPS, URLs, Paths)
Primary Focus	Connecting internal network components to Pods	Exposing HTTP/HTTPS routes from outside to internal Services
Routing Intelligence	None. Forwards all traffic blindly from a port to a Pod	High. Can route based on domain names (host) and URLs (path)
SSL/TLS Termination	No (must be handled by your application code inside the Pod)	Yes (centralized management of SSL certificates at the routing layer)
Cost Efficiency	**Expensive at scale:** Every LoadBalancer service creates a new cloud load balancer ($$$)	**Cost-efficient:** You can expose dozens of services behind a single cloud load balancer running the Ingress Controller

Kubernetes Architecture:

Container Runtime: A container runtime is a low-level component of a container engine that mounts the container and works with the OS kernel to start and support the containerization process. Ex: Docker, Windows Containers, runC.

Docker: It is a container technology which allows applications to be packaged as containers.

- It is also deployed on every node.



***DEF:** Kubernetes is a powerful container orchestration platform that automates the deployment, scaling, and management of containerized applications. Its architecture is*

designed to be scalable, resilient, and highly available. Here's an overview of the key components and architecture of Kubernetes:

1. **Master Node:** The master node is responsible for managing the cluster and orchestrating operations. It typically consists of several components:
   - **kube-apiserver:** Exposes the Kubernetes API, which allows clients to interact with the cluster. And also manages the internal communication. **The gateway (The only one that talks to etcd).**
     - **The kube-controller-manager never talks to etcd directly.**
     - Only the kube-apiserver is allowed to talk to etcd. Every other component in Kubernetes must talk to the API Server instead.
   - **etcd:** (Kubernetes memory store) A distributed key-value store that stores the cluster's state, including configuration data, metadata, and other critical information. **The source of truth (Database).**
   - **kube-scheduler:** Assigns pods (groups of one or more containers) to nodes based on resource requirements, constraints, and availability.
   - **kube-controller-manager:** Manages various controllers that regulate the state of the cluster, such as node controllers, replication controllers, and endpoint controllers. **The logic (Making sure count is right).**
2. **Worker Nodes (Minions):** Worker nodes are the compute resources where containers are deployed and executed. Each worker node runs the following components:
   - **kubelet:** Agent that runs on each node and is responsible for managing containers, handling container lifecycle events, and reporting node status to the master node. **The hands (Starting the actual containers on the Node).**
   - **kube-proxy:** Network proxy that maintains network rules and forwards traffic to the appropriate containers or services.
   - **Container Runtime:** Software responsible for running containers, such as Docker, containerd, or cri-o.
3. **Pods:** Pods are the smallest deployable units in Kubernetes cluster. **A layer of abstraction on top of containers**. Containers within a pod share the same network namespace and can communicate with each other using localhost. A pod usually carry a single application, means every container will hold the same application, acts as replicas.
   1. **Abstraction:** A process of hiding complex implementation details or strategies.
4. **Deployment:** A Kubernetes resource, **A layer of abstraction on top of pods**. Which makes easy access to interact with pods, replicate and other configurations.
   1. It works same as MIG for compute engines.
   2. A Kubernetes **Deployment** is a resource that functions similarly to a Google Cloud **MIG (Managed Instance Group)**. While a MIG manages virtual machines, a Deployment manages containers (Pods)—providing automated scaling, rolling updates, and self-healing features."
5. **Services:** A kubernetes resource acts like a **Load Balancer** or a middleman. When a request hits the Service, it looks at the "labels" on your Pods and forwards the traffic to one that is healthy and available.
   - In Kubernetes, Pods are "ephemeral"—meaning they die and get replaced constantly. Every time a new Pod is born, it gets a new IP address. If your frontend tried to talk directly to a backend Pod's IP, it would lose the connection the moment that Pod restarted.
   - A **Service** solves this by providing a single, stable IP address and DNS name that never changes, even if the Pods behind it do.
6. **ReplicaSets:** A resource which ensure that a specified number of identical pod replicas are running at any given time. They are used to maintain the desired state of pods and ensure high availability and scalability of applications.
7. **Volumes:** Volumes provide persistent storage for containers and allow data to be

shared between containers within the same pod. Kubernetes supports various types of volumes, including emptyDir, hostPath, persistentVolumeClaim, and cloud-based storage solutions.

8. **Imperative**: Giving specific, step by step commands to perform an action. (eg, create this pod now)(EX: kubectl **edit, create, run**)
9. **Declarative**: Final desired state in a file, letting Kubernetes handle how to achieve it. (Eg: This is how I want the cluster look like) (Ex: **kubectl apply**)

ETCD:

A distributed key-value store that holds cluster state.

- The leader is responsible for coordinating all write requests and ensuring data is replicated across the cluster using the **Raft consensus algorithm**.
  - **Raft consensus algorithm** is a protocol designed to ensure that a cluster of nodes can agree on a shared state, even if some nodes fail. It requires **Quorum** to maintain.
  - **Quorum** is the minimum number of nodes that must be online and in agreement for the cluster to perform any operations. **Q=[n/2]+1** (EX: For 3 node – 2 must be up & running)
- Identify the leader is by using the **etcdctl** command-line tool.

Troubleshooting commands:

Purpose	Command	Notes
Check health of all ETCD nodes	`etcdctl endpoint health --cluster`	All nodes should return **healthy: true**. Used to verify cluster health.
Identify ETCD Leader	`etcdctl endpoint status --cluster -w table`	Shows leader, raft term, database size, and endpoint status in a table format.
List ETCD members	`etcdctl member list -w table`	Displays all ETCD members, their IDs, peer URLs, client URLs, and state.
Identify Patroni Leader	`patronictl -c /etc/patroni/patroni.yml list`	Displays PostgreSQL cluster members, leader, replicas, and replication status.
Check PostgreSQL Leader stored in ETCD	`etcdctl get /service/my_cluster/leader`	Retrieves the current Patroni leader information stored in ETCD. Replace my_cluster with your cluster name.
Remove dead Patroni member entry	`etcdctl del /service/my_cluster/members/node_name`	Removes stale member metadata when a node is permanently removed and Patroni doesn't clean it automatically.
Force Leader Reset	`etcdctl del /service/my_cluster/leader`	Deletes the leader key, forcing Patroni to perform a new leader election. Use only during recovery scenarios.

Identify ETCD Leader (TLS Enabled)	`bash\netcdctl \\\n --endpoints=https://<NODE_IP>:2379 \\\n --cacert=/path/to/ca.crt \\\n --cert=/path/to/client.crt \\\n --key=/path/to/client.key \\\n endpoint status\n`	Required when ETCD client authentication is secured with TLS certificates.
Configure ETCDCTL environment variables (TLS)	`bash\nexport ETCDCTL_API=3\nexport ETCDCTL_ENDPOINTS= \"https://10.0.1.10:2379,https://10.0.1.11:2379,https://10.0.1.12:2379\"\nexport ETCDCTL_CACERT=\"/etc/etcd/pki/ca.pem \"\nexport ETCDCTL_CERT= \"/etc/etcd/pki/client.pem\"\nexport ETCDCTL_KEY=\"/etc/etcd/pki/client-key.pem\"\n`	Sets default TLS parameters so every command doesn't require certificate options.
Check ETCD status after setting environment variables	`etcdctl endpoint status -w table`	Simplified command after exporting TLS environment variables.
Check Patroni Leader after setting environment variables	`etcdctl get /service/my_cluster/leader`	Retrieves leader information without specifying certificates repeatedly.
Delete ETCD data (Dangerous)	`etcdctl --endpoints=${ENDPOINTS} del --prefix /db/`	**Stop Patroni on all database nodes before running.** Deletes all keys under the specified prefix and is typically used only during cluster rebuild or recovery.

If not stopped patroni – patroni will think an accidental failover. Even if your PostgreSQL nodes are perfectly healthy, they will stop being "Primary" if etcd is broken. This is why etcd is called the Source of Truth.

1. What happens if etcd leadership fails?

- If etcd loses its own leader (e.g., 2 out of 3 etcd nodes go down), it can no longer reach a quorum.
  - **etcd** goes read-only or becomes unresponsive.
  - **Patroni gets worried:** It can no longer update its "Leader Key" in etcd.
  - **The Safety Trigger:** To prevent data corruption, Patroni will usually **demote** the PostgreSQL Primary to a Secondary.

2. What is "Updating the Leader Key"?

- Think of the **Leader Key** like a digital "lease" on an apartment. In etcd, the leader key (e.g., /service/my_cluster/leader) has a **TTL (Time To Live)**, usually set to 30 seconds.
- **The Continuous Update:** Patroni must "renew" this lease every few seconds (usually every 10 seconds). This renewal is what we mean by **updating the leader key**.
- **The Heartbeat:** If Patroni successfully updates the key, it tells the rest of the cluster: *"I am still alive and healthy, do not hold an election."*

Personal experience: I've worked with etcd in HA database setups, so I understand distributed

key value stores and consistency, which directly applies to kubernetes control plane.

Local ENV of K8's:

MiniKube**:**

- Minikube is a popular tool for running a single-node Kubernetes cluster locally on a developer's machine.
- It is to test a deployment on local machine, it creates Virtual Box on your laptop.
- Node runs in that virtual box
- Node K8s cluster
- For testing purposes.

Kubectl:

- It is a command-line interface (CLI) tool for working with a Kubernetes cluster.

Kubectl commands:

List the pods:	`kubectl get pods`
Details about a specific pod:	`kubectl describe pod <pod-name>`
	`Kubectl describe deployment <deployment-name>`
List all pods in a specific namespace:	`kubectl get pods -n <namespace>`
	`Kubectl get deployments`
	`Kubectl get services`
Create a new resource	`Kubectl create -f pod.yaml`
	`Kubectly create deployment nginx --image=nginx`
Apply changes to a resource from a file	`Kubectl apply -f pod.yaml`
	`Kubectl apply -f deployemt.yaml`
Delete a pod:	kubectl delete pod <pod-name>
	Kubectl delete deployment <deployment-name>
Get logs from a specific pod:	`kubectl logs <pod-name>`
Follow logs from a specific pod in real-time:	`kubectl logs -f <pod-name>`
Execute a command in a running pod:	`kubectl exec -it <pod-name> -- <command>`
Ex:	`kubectl exec -it <pod-name> -- /bin/bash`
Port forwarding to a specific port in a pod:	`kubectl port-forward <pod-name> <local-port>:<pod-port>`

- **<u>Difference between create and apply command:</u>**
  - Create command will create a new resource, if the resource already exists it will return an error.
  - Apply command will create and update resources based on the configuration provided. If the resource is already exists apply will update the resource with the new configuration.

YAML Configuration file:

Version2 - Skip it, or read for detailed instructions

```yaml
# 1. THE API VERSION & KIND
# Defines the schema we are using. 'apps/v1' is the stable version for
Deployments.
apiVersion: apps/v1
kind: Deployment
# 2. METADATA
# Unique identification for the Deployment object itself.
metadata:
  name: pro-webapp-deployment  # The name of the manager
  namespace: production        # Logical isolation for the app
  labels:                      # Tags for organizing/filtering objects
    tier: frontend
    env: prod
# 3. SPEC (The Deployment's Strategy)
# This section defines "How" the manager should behave.
spec:
  replicas: 3                  # Desired number of pods (The "Goal")
```

```yaml
  # How long to wait after a pod is 'Ready' before marking the rollout as
successful.
  minReadySeconds: 10

  # Keeps the last 5 versions in history so you can run 'rollout undo'.
  revisionHistoryLimit: 5
  # SELECTOR: The "Glue"
  # This tells the Deployment: "I am responsible for all pods that have these
labels."
  selector:
    matchLabels:
      app: my-cool-app
  # STRATEGY: How to handle updates.
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1              # Can create 1 extra pod during update (Total 4)
      maxUnavailable: 0        # Never allow pods to be down; always stay at 3
  # 4. TEMPLATE (The Blueprint for the Pod)
  # This is essentially a "Pod definition" nested inside the Deployment.
  template:
    metadata:
      labels:                  # MUST match the selector.matchLabels above!
        app: my-cool-app

    # 5. POD SPEC
    # Defines the actual containers and their requirements.
    spec:
      containers:
      - name: main-container
        image: nginx:1.21.1
        imagePullPolicy: IfNotPresent # Only download image if not on the Node

        # PORTS: Informational, doesn't actually open the port (that's for
Service).
        ports:
        - containerPort: 80
        # RESOURCES: Critical for Senior Devs to prevent cluster crashes.
        resources:
          requests:            # Guaranteed resources (What the pod needs to
start)
            memory: "64Mi"
            cpu: "250m"
          limits:              # Max allowed (K8s kills pod if it exceeds memory
limit)
            memory: "128Mi"
            cpu: "500m"
        # HEALTH CHECKS (PROBES)
        # Liveness: "Am I alive?" (If fails, K8s restarts the pod)
        livenessProbe:
          httpGet:
            path: /healthz
            port: 80
          initialDelaySeconds: 15
          periodSeconds: 20
        # Readiness: "Am I ready for traffic?" (If fails, Service stops sending
traffic)
        readinessProbe:
          httpGet:
            path: /ready
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 10
```

```
        # ENVIRONMENT VARIABLES
        env:
        - name: DB_URL
          value: "mydb.production.svc.cluster.local"
```

Deep Dive into Key Sections:

The selector vs. template.metadata.labels

This is where 90% of beginners fail. The selector is the "Search Query" the manager uses. If the labels in the template don't match the selector, the Deployment will continuously try to create pods because it can't "find" the ones it just made.

resources (Requests vs. Limits)

- **Requests:**
  - This is the **minimum** amount guaranteed to the container. Kubernetes uses this number to decide which node has enough room to "fit" the pod.
  - Used by the **Scheduler**.
  - If a node has 1GB RAM and your request is 2GB, the pod won't go there.
- **Limits:**
  - This is the **maximum** amount the container is permitted to use. It prevents a container from becoming a "noisy neighbor" that steals resources from other pods.
  - Used by the **Kubelet**.

Resource	If the Limit is Reached...	Behavior Type
CPU	The container is **Throttled** (slowed down). It is not killed, but it gets less processing time.	Compressible
Memory	The container is **Terminated** with an **OOMKilled** (Out of Memory) error. It is then restarted based on the policy.	Incompressible

livenessProbe vs. readinessProbe

- **Liveness:** Fixes "Frozen" apps. If your app code hangs, this probe fails, and K8s kills the pod to start a fresh one.
- **Readiness:** Fixes "Traffic issues." If your app is still loading a heavy database during startup, it isn't "Ready." K8s will wait until this passes before sending users to that pod.

The Three Layers of Scaling

In production, these three work together in a hierarchy.

Layer 1: Horizontal Pod Autoscaler (HPA)

- **What it scales:** The number of **Replicas** (Pods).
- **The Logic:** "If CPU usage > 70%, add another Pod."
- **Best for:** Stateless web servers and APIs.
- **Production Tip:** Always define **Resource Requests**. HPA calculates percentages based on what you *requested*, not the limit. If requests are missing, HPA can't calculate accurately.

Layer 2: Vertical Pod Autoscaler (VPA)

- **What it scales:** The **Resources** (CPU/Memory) of a Pod.
- **The Logic:** "This Pod is always using 2GB of RAM but only requested 512MB. Let's restart it with 2.5GB."
- **The Catch:** VPA usually requires a **Pod restart** to apply new limits.
- **Production Tip:** Use VPA in "Recommendation Mode" (updateMode: "Off") first. It will tell you

what your Pods *actually* need without killing them. This is great for "right-sizing" your cluster to save costs.

- **VPA cannot be used without Custom Resource Definitions (CRDs)** because it is not part of the "core" Kubernetes binary. While objects like Pods, Deployments, and Services are compiled directly into the Kubernetes source code, the **Vertical Pod Autoscaler** was developed as an optional add-on.

Task/Resource	HPA Command	VPA Command
Formatting Style	**Imperative**	**Declarative**
Creation	kubectl autoscale deployment [name] --cpu-percent=50 ...	kubectl apply -f vpa-definition.yaml
Check Status	kubectl get hpa	kubectl get vpa
Details	kubectl describe hpa	kubectl describe vpa
What it Changes	spec.replicas	resources.requests/limits

Layer 3: Cluster Autoscaler (CA)

- **What it scales:** The number of **Nodes** (Virtual Machines).
- **The Logic:** "I have 5 new Pods from the HPA, but no Node has enough free RAM to run them. Let's ask AWS/Azure/GCP for another VM."
- **Production Tip:** CA scales based on **Pending Pods**, not CPU usage. If a Pod can't fit anywhere, CA kicks in.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend-service
  minReplicas: 2      # Always have at least 2 for High Availability
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60 # Scale up if average CPU hits 60%
```

Senior Interview Strategy: "The Conflict"

A favorite senior-level interview question: **"Can you run HPA and VPA together?"**

- **The Wrong Answer:** "Yes, you just apply both."
- **The Senior Answer:** "Generally, **no**, if they are scaling on the same metric (like CPU). If HPA is trying to add more Pods because CPU is high, and VPA is trying to make the Pod bigger because CPU is high, they will fight each other, leading to cluster instability."
- **The Exception:** "You can use them together if HPA uses a **custom metric** (like 'Number of Requests per second') while VPA manages the CPU/Memory."

Labels and Selectors:

They connect deployments to pods.

- Pods get the label through the template blueprint.
- This label is matched by the selector.

Labels: It is like a discovery, they are key value pairs that can attach to objects like pods.

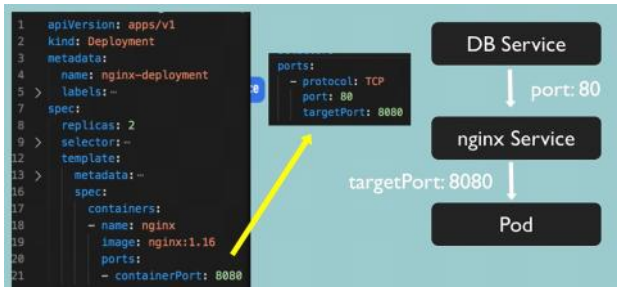
- Metadata part contains labels

Selectors: Spec block contains selector, It connects the deployment with the pod.

```
selector:
  matchLabels:
    app: nginx
```

Annotations: They are to attach metadata to objects in the form of key-value pairs.

Ports in Service and Pod:



- targetPort of the service need to match the containerPort inside pod's specification.

ConfigMaps:

The Core Principle: Separation of Concerns

In a production environment, your application consists of two distinct parts:

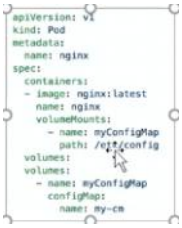
1. **The Artifact (Image):** The logic/binary. It is **Immutable** (never changes).
2. **The Context (Config):** The environment-specific data (URLs, Keys, Flags).

ConfigMaps and Secrets act as the "Bridge" that connects these two at runtime.

A ConfigMaps are API object used to store non-confidential data in key-value pairs.

- ConfigMaps mainly used to store configurations rather than changing in the application by modifying, building and pull the image again.
- Pods can consume ConfigMaps as environment variables, command-line arguments, or as configuration files in a volume.

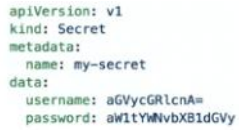
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-cm
data:
  name: mydata.txt
  contents: |
    you can store
    multiline content
    and configfiles
```

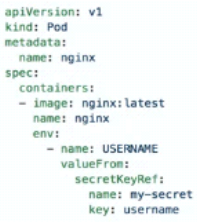


Secrets:

A Secret is an object that contains a small amount of sensitive data such as a password, a token, or a key.

- Such information might otherwise be put in a Pod specification or in a container image.
- Using a Secret means that you don't need to include confidential data in your application code.






```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-auth
type: Opaque
data:
  # Values must be base64 encoded!
  password: cGFzc3dvcmQxMjM= # "password123"
---
# Inside the manifest:
env:
- name: DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: db-auth
      key: password
```

Realtime example:

Scenario 1: The "Anti-Pattern" (Building Multiple Images)

In this scenario, the configuration is hardcoded inside the application directory. Because the code is different for each environment, the Docker compiler sees a "change" and must create an entirely new image for each.

1. The Code Folders

- **Folder /dev:** contains config.json → { "db_url": "dev.db.com" }
- **Folder /prod:** contains config.json → { "db_url": "prod.db.com" }

2. The Dockerfile

```dockerfile
FROM node:18
```

```
WORKDIR /app
COPY . .
# The config.json is "baked" into the image here.
RUN npm install
CMD ["node", "server.js"]
```

3. The Build Command (The Problem)

You are forced to run the build command **twice**:

- docker build -t my-app:dev ./dev
- docker build -t my-app:prod ./prod

Result: You have two different images. If the "Dev" image passes QA, you still can't be 100% sure the "Prod" image will work because it is a **physically different file**.

Scenario 2: The "Production-Ready" Way (Building Once)

In this scenario, we use **Environment Variables** in the code. The image becomes "Hollow"—it contains the logic but not the data.

1. The Code (server.js)

Instead of reading a local file, the code looks at the Operating System's environment variables.

```
// server.js
const dbUrl = process.env.DATABASE_URL; // Kubernetes will inject this!
console.log(`Connecting to: ${dbUrl}`);
```

2. The Dockerfile

Notice there are no environment-specific files copied here.

```
FROM node:18
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "server.js"]
```

3. The Build Command (The Benefit)

You build **one time** and tag it with a version:

- docker build -t my-app:v1.0.0 .

4. The Kubernetes Connection (The "Magic")

Now, you take that **single image** (v1.0.0) and use Kubernetes ConfigMaps to tell it where to point, depending on where it is running.

For Development:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: dev
data:
  DATABASE_URL: "dev.db.com"
```

For Production:

```
apiVersion: v1
kind: ConfigMap
metadata:
```

```
  name: app-config
  namespace: prod
data:
  DATABASE_URL: "prod.db.com"
```

The Deployment (Same for both!)

```
spec:
  containers:
  - name: my-app
    image: my-app:v1.0.0  # The EXACT same image in both namespaces
    env:
    - name: DATABASE_URL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: DATABASE_URL
```

PERMISSIONS(RBAC):

Manage permission k8s cluster permission, which object need to manage:

1. The Four Pillars of RBAC

To set up a permission, you need to answer: **Who** is doing **What** to **Which** resource?

- **Subject (Who):** A User, a Group, or a **ServiceAccount** (this is what your "Cluster Manager" pod would use to talk to the K8s API).
- **Resources (Which):** Pods, Services, Nodes, or your **Custom CRDs** (like DBCluster or Backup).
- **Verbs (What):** The actions: get, list, watch, create, update, patch, delete.
- **Role / RoleBinding:** The "Rulebook" that connects the Who, What, and Which.

Service Account:

- Service accounts are used to authenticate and authorize pods running within the cluster. Each pod runs with a specific service account, and you can grant permissions to these service accounts using Role-Based Access Control (RBAC).

2. Role vs. ClusterRole

This is the most important distinction for a Senior Engineer.

- **Role:** Permissions within a **specific Namespace**.
  - *Example:* Allow a developer to see logs only in the alloydb-prod namespace.
- **ClusterRole:** Permissions across the **entire Cluster**.
  - *Example:* Allow an administrator to view all Nodes or manage your Backup CRDs across every namespace.

3. RoleBinding vs. ClusterRoleBinding

This is how you "attach" a person to a set of rules.

- **RoleBinding:**
  - RoleBindings bind a Role to a specific user, group, or service account within a namespace. They grant permissions defined by the Role to the subjects specified in the binding.
- **ClusterRoleBinding:**

- ClusterRoleBindings bind a ClusterRole to a user, group, or service account at the cluster level. They grant permissions defined by the ClusterRole to the subjects specified in the binding.

4. Useful Commands for RBAC

When you are troubleshooting "Permission Denied" errors, use these commands:

```
# Check if I can do a specific action (The "Can-I" command)
kubectl auth can-i create deployments --namespace alloydb-prod

# Check if a specific ServiceAccount can do something
kubectl auth can-i get pods --as=system:serviceaccount:default:cluster-manager

# List all Roles and Bindings
kubectl get roles,rolebindings,clusterroles,clusterrolebindings -A
```

Difference between GCP Level (IAM) & RBAC:

- **Analogy:** If your Kubernetes cluster is a high-security building, **GCP IAM** is the guard at the front gate who lets you into the parking lot, but **RBAC** is the internal security badge system that decides which specific rooms (Namespaces) you can enter and which buttons (API actions) you can press once you are inside.

- **GCP Level (IAM):** Your VM has a **GCP Service Account** attached to it. This allows the VM to upload backups to a **GCS Bucket**. If you don't give the storage.admin role in the GCP Console, the VM can't see the bucket.

- **Inside the App (RBAC):** Now, imagine you have a **Cluster Manager** process running on that VM. You need a way to tell the Cluster Manager: *"You are allowed to see the Postgres logs, but you are NOT allowed to delete the Database."*

Types of scope permissions(Global/more specific):

In Kubernetes, permissions can be scoped at different levels of specificity, ranging from global to more specific scopes. Here are the main types of scopes for permissions:

Global Scope:

- Global scope permissions apply across the entire Kubernetes cluster, affecting all namespaces and resources within the cluster.
- Examples include **ClusterRoles** and **ClusterRoleBindings**, which define permissions that apply cluster-wide.

Namespace Scope:

- Namespace scope permissions apply to resources within a specific namespace in the Kubernetes cluster.
- Examples include **Roles** and **RoleBindings**, which define permissions that apply within a single namespace.

Resource Scope:

- Resource scope permissions apply to specific types of Kubernetes resources, such as **Pods**, **Deployments**, **Services**, **ConfigMaps**, **Secrets**, etc.

- Roles and ClusterRoles can be scoped to specific resources, defining permissions that apply only to those resources.

Verb Scope:

- Verb scope permissions define specific actions that are allowed or denied on resources.
- Examples of verbs include **get, list, watch, create, delete, update, patch**, etc.
- Roles and ClusterRoles specify which verbs are permitted or denied for each resource they apply to.

Subject Scope:

- Subject scope permissions define who (or what) is granted or denied access to resources.
- Subjects can be **users, groups, or service accounts.**
- RoleBindings and ClusterRoleBindings associate roles or cluster roles with specific subjects, granting them the permissions defined in the roles.

Network policies:

A **Network Policy** is a set of rules in Kubernetes that controls how pods are allowed to communicate with each other and with other network endpoints.
By default, Kubernetes uses a "flat" network where every pod can talk to every other pod. Network Policies allow you to change this to a "Least Privilege" model, where you explicitly define which traffic is allowed.

Layers of Abstraction:

- Deployment manages a ReplicaSet.
- ReplicaSet manages a Pod.
- Pod is an abstraction of a Container.
- Container is the fundamental unit.

Namespaces:

- Resource separation is possible by Namespaces in K8's

It provides a mechanism for isolating groups of resources within a single cluster.
Namespaces are a way to divide cluster resources between multiple users.

- Kubernetes supports multiple virtual clusters backed by the same cluster. These virtual clusters are called namespaces.
- Namespace-based scoping is applicable only for namespaced objects (e.g. Deployments, Services, etc) and not for cluster-wide objects (e.g. StorageClass, Nodes, PersistentVolumes, etc).

Object creation	Usage

`apiVersion: v1`
`kind: Namespace`
`metadata:`
`  name: billing-prod`
`  labels:`
`  environment: production`
`  team: finance`
	`apiVersion: v1`
	`kind: Pod`
	`metadata:`
	`  name: payment-processor`
	`  namespace: billing-prod # <--- This maps the pod into that specific boundary`
	`spec:`
	`  containers:`
	`  - name: web`
	`  image: nginx:latest`

- There are 4 default namespaces:
  - **Kube-system:**
    - **Purpose:** This namespace is used for system resources and components managed by Kubernetes itself.
    - **Use Case:** Kubernetes keeps its own components (like the API server and scheduler). **Don't touch things here!**It's critical for the functioning of the cluster.
  - **Kube-public:**
    - **Purpose:** This namespace is readable by all users (including those not authenticated). It's typically used for public resources.
    - **Use Case:** It can contain public, cluster-wide information. For example, you might find a ConfigMap in this namespace that holds cluster information that needs to be accessible to everyone.
  - **Kube-node-lease:**
    - **Purpose:** This namespace is used for node leases, which are heartbeats that nodes send to indicate they are still running.
    - **Use Case:** It helps in determining the health of nodes in the cluster. Node leases in this namespace improve the efficiency of the node heartbeats mechanism.
  - **Default:**
    - This is the default namespace for objects with no other namespace.
    - **Use Case**: If you don't specify a namespace when creating resources, they are placed in the default namespace. It's a convenient place for small clusters or simple setups where namespaces aren't heavily used.

What are node-leases:

Node Leases are small, lightweight objects that act as a "heartbeat" for your cluster. They are the way a node tells the master (Control Plane), "I am still alive and healthy!"

```
apiVersion: coordination.k8s.io/v1
kind: Lease
metadata:
  name: node-01
  namespace: kube-node-lease
spec:
  holderIdentity: node-01         # The name of the node
  leaseDurationSeconds: 40        # How long the lease is valid
  renewTime: "2026-04-27T16:45:01Z" # The last time the node said "I'm
alive"
```

3. Why Use Them? (The 3 Main Benefits)

A. Environment Separation

- Most companies use namespaces to separate stages of a project:
  - project-alpha-dev

- project-alpha-staging
- project-alpha-prod

B. Multi-Tenancy

- If you are a DevOps engineer supporting three different teams (Marketing, Sales, Engineering), you give each team their own namespace. This prevents the Marketing team from accidentally deleting the Sales team's database.

C. Resource Quotas

- You can set a "budget" for a namespace. For example, you can tell Kubernetes: *"The 'Test' namespace can never use more than 4 CPUs."* This is called a **ResourceQuota**.

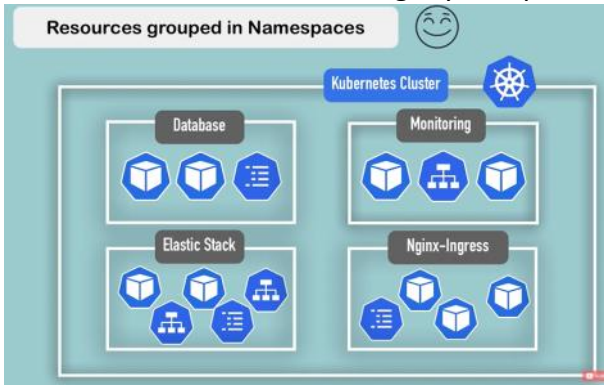
They can't do:

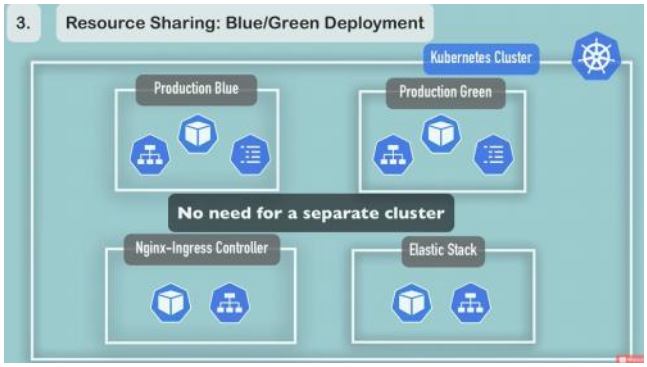
- **They aren't physical:** Pods from different namespaces can still run on the same physical node.
- **They aren't a firewall (by default):** By default, a Pod in the "Dev" namespace **can** still talk to a Pod in the "Prod" namespace via its IP address. If you want to block that, you need to use **Network Policies**.
- **Some things are "Global":** Not everything lives in a room. Nodes, PersistentVolumes, and the Namespaces themselves are "Cluster-wide" objects.

Troubleshooting:

- **To get current namespaces:** `kubectl get namespaces`
- **To create new namespace:** `kubectl create namespace my-new-app`
- **To run a pod in a specific namespace:** `kubectl run nginx-pod --image=nginx --namespace=my-new-app`

- **To switch your "default" view:** If you get tired of typing --namespace every time, you can change your context (using a tool like kubens or the standard command) so that all commands target your specific room by default.





Volumes:

- In Kubernetes, Pods are **ephemeral** (temporary).
- If a Pod dies or is restarted, any data inside its container is wiped clean—gone forever. To save your data, you need to plug in a Volume.
- They are used to store some data in a directory that can be accessible across the containers running in a pod.
- Kubernetes will not take care of storage maintenance.
- It is independent on pod lifecycle, cluster crash and namespaces (They are not belonging to any namespace).

Mainly uses 3 types of Volumes:

3. **emptyDir :**
   - **Description:** The **emptyDir** is a volume that is created when a Pod is assigned to a Node, and it exists as long as that Pod is running on that node.
     - EmptyDir is physically stored on the disk of the Node. Gets deleted once pod deleted. (**/var/lib/kubelet/pods/...**)
     - The physical data is sitting safely on the hosting **Node's** hard drive, it gets wiped out because **emptyDir is legally tied to the lifecycle of the Pod, not the Node.**
     - **Analogy** : emptyDir is a "**Private Safe**" inside a room in a hotel.
   - **Step-by-Step: How it works**
     - **Pod Starts:** Kubernetes creates a blank folder on the Host machine (the Server).
     - **Mounting:** Kubernetes "plugs" that folder into your container at a specific path (e.g., /data).
     - **Container Crash:** If your app crashes and the container restarts, the new container plugs back into the **same folder**. Your data is still there!
     - **Pod Deletion:** If you delete the Pod, Kubernetes deletes the folder on the server. Data is gone.
   - **Use Case:**
     - **Checkpointing:** If your app takes a long time to calculate something and you don't want to restart from zero if the app crashes.
     - **Shared Space:** If you have two containers in the same Pod (e.g., one downloads a file, the other processes it), they can both "plug into" the same emptyDir to share the file.

```
apiVersion: v1
kind: Pod
metadata:
  name: storage-pod
spec:
  containers:
  - name: my-app
```

```
    image: nginx
    volumeMounts:
    - name: scratch-space
      mountPath: /data    # Where the app sees the folder
  volumes:
  - name: scratch-space
    emptyDir: {}          # This tells K8s to create a blank temporary
folder
```

4. **Hostpath volume:**
   - **Description:** A hostPath volume mounts a specific file or directory from the **Host Node's** (the physical server's) hard drive directly into your Pod.
     - **Analogy** : **hostPath** is like a **Shared Cabinet** in the **Building Lobby.**
     - **Hostpath** is physically stored on the disk of the Node. The data **stays on the disk** forever even pod is deleted.
   - **Step-by-Step: How it works**
     - **Preparation:** A folder already exists on the actual Server (the Node), like /var/logs or /etc/config.
     - **Mounting:** You tell Kubernetes, "Take that folder from the Server and 'map' it into my Pod at /app/logs."
     - **Pod Deletion:** If the Pod is deleted, the folder on the Server **remains exactly as it is**.
     - **Rebirth:** If a new Pod starts **on the same Node**, it plugs back into that folder and sees all the old data.
   - **Use Case:**
     - **System Tools:** If you are running a monitoring tool (like Fluentd or Prometheus) that needs to read the server's own logs to see if it's healthy.
     - **Performance:** It's very fast because it's a direct link to the local hard drive.
     - **Development:** Sometimes used in "Minikube" or local setups to sync your code from your laptop into the cluster.

```
apiVersion: v1
kind: Pod
metadata:
  name: hostpath-pod
spec:
  containers:
  - name: log-reader
    image: busybox
    volumeMounts:
    - name: server-logs
      mountPath: /data/logs    # Where the app sees it inside the container
  volumes:
  - name: server-logs
    hostPath:
      path: /var/log           # The REAL folder on the physical Server
      type: Directory          # Ensures the path must be a directory
```

     - All files present inside **/var/log** will be visibile by **/data/logs** folder inside container

5. **Persistent Volumes: Persistent Volume (PV)** as an **External Hard Drive** that exists independently of any single computer.
   - **How it works: The "Two-Step" Process**
     - In Kubernetes, managing external storage is a conversation between the **Administrator** and the **Developer**. This is why we have two parts:

- **Persistent Volume (PV):** This is the "Physical" (or Cloud) storage resource created by an Admin. It says: *"I have a 10GB disk available on AWS/GCP."*
- **Persistent Volume Claim (PVC):** This is the "Request" made by the Developer. It says: *"I need 5GB of storage for my database."*

- **Step-by-Step: The Lifecycle**
  - **Provisioning:** An Admin creates a **PV**. It's just sitting there, "Available."
  - **Binding:** A Developer creates a **PVC**. Kubernetes looks at the available PVs and says, *"Hey, this 10GB PV matches your 5GB request. I'll link them together."*
  - **Using:** The Pod uses the **PVC** as a volume.
  - **Reclaiming:** When the Pod is deleted, the **PVC** can stay. If the PVC is deleted, the **PV** is either wiped, kept, or deleted based on a "Reclaim Policy."

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-storage
spec:
  capacity:
    storage: 10Gi         # Size of the "Hard Drive"
  accessModes:
    - ReadWriteOnce       # Only one Pod can write at a time
  persistentVolumeReclaimPolicy: Retain # Keep data if the claim is
deleted
  hostPath:
    path: "/mnt/data"     # In real life, this would be a Cloud Disk
like AWS EBS
```

Parameters:

a. Capacity
b. Access modes:
   - ReadOnlyMany (ROX)
   - ReadWriteOnce(RWO) - This means the storage volume can only be mounted by **one single node** at a time.
   - ReadWriteMany(RWX) - This means the storage can be mounted by **many nodes** simultaneously.
c. Persistent Volume Reclaim Policy
   - Retain
   - Recycle
   - Delete
d. StorageClass

- **Persistent Volumes Claims:** A **Persistent Volume Claim (PVC)** is a request for storage by a user. It doesn't care *where* the storage comes from; it only cares about **how much** and **how fast.**
  - Claims must be in the same namespace.
  - **Step-by-Step: How to use it in a Pod**
    - **Step A:** The Developer makes the Claim (The Ticket)

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-database-pvc
spec:
  accessModes:
    - ReadWriteOnce
```

```
    resources:
      requests:
        storage: 5Gi    # "I need 5 Gigabytes, please!"
```

- **Step B:** The Developer "plugs" the Ticket into the Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: database-pod
spec:
  containers:
    - name: mysql
      image: mysql
      volumeMounts:
        - name: db-storage
          mountPath: /var/lib/mysql
  volumes:
    - name: db-storage
      persistentVolumeClaim:
        claimName: my-database-pvc  # This connects the Pod to
the Voucher
```

Why do we split them up? (The "Abstraction" Power)

You might ask: *"Why not just define the storage directly in the Pod?"*

1. **Portability:** You can take the exact same Pod YAML and run it on AWS, Google Cloud, or your own laptop. You don't have to change the Pod code; you just ensure a "Claim" exists in each environment.
2. **Security:** Developers don't need to know the "Root Passwords" or IP addresses of the massive storage servers. They just ask for "5GB," and the Admin handles the messy details behind the scenes.
3. **Independence:** If the Pod crashes or you delete it to update your app, the **PVC remains**. When the new Pod starts up, it uses the same PVC and instantly gets all the old data back.

6. Storage class:

- It is the "blueprint" that allows Kubernetes to **automatically create storage** (Dynamic Provisioning) on-demand.

The Analogy: The "Beverage Vending Machine"

Imagine you are in an office.

- **Manual Way (PV/PVC):** You want a coffee. You have to find the Office Manager (Admin), ask them to go to the store, buy a specific bag of beans, brew it, and put it on a shelf (PV). Then you can go and take it (PVC).
- **StorageClass Way:** There is a **Vending Machine** in the lobby. You walk up to it, insert your "Order" (PVC), and press a button. The machine grinds the beans and pours the coffee into your cup **on the spot**.

Different Types of Storage

- **"Fast" Class:** Uses expensive SSDs for databases.
- **"Slow" Class:** Uses cheap hard drives for long-term backups.
- **"Replicated" Class:** Uses storage that is copied across three cities for safety.

Step-by-Step: How it works (The Magic)

- **The Admin** creates a StorageClass once. It defines the "recipe" (e.g., "Use AWS SSDs in the US-East region").
- **The Developer** creates a PVC and simply says: storageClassName: **fast-ssd**.
- **The Provisioner** (the "brain" behind the machine) sees the request, calls the Cloud

Provider (AWS/GCP), and says: *"Create a 10GB SSD right now!"*

- **The Link:** Kubernetes automatically creates the **PV** and binds it to your **PVC**.

    - **The Admin's "Recipe" (StorageClass)**

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd-storage
provisioner: kubernetes.io/aws-ebs  # The "Chef" who knows how to talk
to AWS
parameters:
  type: gp2                         # Use SSD-tier storage
```

    - **The Developer's "Order" (PVC)**

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-db-claim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast-ssd-storage # Point to the "Menu Item"
  resources:
    requests:
      storage: 20Gi
```

Volume / Component	emptyDir	hostPath	Persistent Volume (PV)	Persistent Volume Claim (PVC)	StorageClass
Survival	**Temporary**.	**Permanent** (on Node).	**Immortal** (Cloud/Net).	**Stable**. Lives until deleted.	**Template**. Defines the rules.
Physical Location	Local Node Disk.	Specific Node Folder.	**Network / Cloud Storage**.	N/A (It is a logical link).	N/A (It is a policy).
Portability	Stuck to Node.	Stuck to Node.	**Mobile**. Follows the Pod.	Mobile. Follows the Pod.	Global for the Cluster.
Analogy	**Hotel Trash Can**.	**Lobby Locker**.	**Portable SSD**.	**The Voucher** for the SSD.	**The Vending Machine**.
Who Creates it?	Kubernetes (Auto).	Developer (Manual).	Admin (or StorageClass).	Developer.	Admin.
Best For...	Caches / Scratch space.	System log reading.	**Databases / Real Data**.	Requesting specific size.	Automating disk creation.

If I want to share file between containers in the same pod, how can I do it?

- To share files between containers within the same pod in Kubernetes, you can use volumes. Volumes provide a way to share data between containers running in the same pod by mounting a shared storage location accessible to all containers.

- Here's how you can set it up using a emptyDir volume, which is a simple volume type that is initialized when a Pod is created and exists as long as that Pod is running:

  Define a volume in your pod specification. This volume will be shared by the containers within the pod.

Example:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: container1
    image: image1
    volumeMounts:
    - name: shared-volume
      mountPath: /shared-data
  - name: container2
    image: image2
    volumeMounts:
    - name: shared-volume
      mountPath: /shared-data
  volumes:
  - name: shared-volume
    emptyDir: {}
```

Mount the Volume in Containers:
Mount the volume in each container within the pod at the desired mount path.
In the example above, both container1 and container2 are mounting the same volume (shared-volume) at the path /shared-data.

Services:

(How to expose to internet ?)

A Service acts like a **Load Balancer** or a middleman. When a request hits the Service, it looks at the "labels" on your Pods and forwards the traffic to one that is healthy and available.

- In Kubernetes, Pods are "ephemeral"—meaning they die and get replaced constantly. Every time a new Pod is born, it gets a new IP address. If your frontend tried to talk directly to a backend Pod's IP, it would lose the connection the moment that Pod restarted.
- A **Service** provides a single, stable IP address and DNS name that never changes, even if the Pods behind it do.
- A **Service** in Kubernetes is an **abstraction that gives a stable network identity (IP address and DNS name) to a set of pods.**

Why do we need it?

- **Service Discovery:** You don't need to keep track of IP addresses; you just point your code to the Service name (e.g., http://backend-service).
- **Load Balancing:** It automatically spreads the work across all available Pods so one doesn't get overwhelmed.
- **Zero Downtime:** When you update your app, the Service ensures traffic only goes to the "Ready" versions of your code.

Services:

1) **Headless Service: -** A **Headless Service** is a specific type of Kubernetes Service that doesn't want to act like a middleman.

   - Normally, a Service gives you a single "ClusterIP" and load-balances traffic for you. A Headless Service says, **"I'm not giving you an IP. Here is a list of the IP addresses of all my Pods—you figure out which one to talk to."**

- You create one by setting `clusterIP: None` in the Service definition.

Key Use Cases

- When client wants to communicate with one specific pod directly or pods want to talk directly with specific pod.
- Stateful example: Database, Stateless: Http request, google search

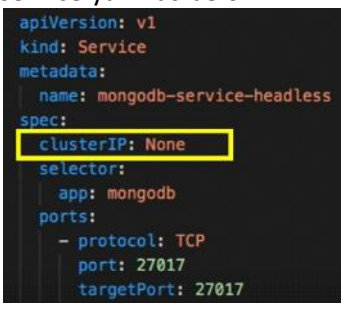
1. Stateful Databases (The #1 Use Case)

In a distributed database (like MongoDB, Cassandra, or RabbitMQ), Pods aren't identical. You usually have a **Primary** (for writes) and **Secondaries** (for reads). (PostgresSql is not distributed database, it scales vertically, individual memory and CPU at a time only one will acts as primary)

- **The Problem:** A standard Service would randomly send your "Write" request to a Secondary Pod, which would fail.
- **The Headless Solution:** It allows the client (or the database nodes themselves) to identify and connect to a specific Pod by its unique DNS name (e.g., `mongodb-0.mongodb-service`).

Client side execution flow:

1. Keep your service.yaml as below:



2. If you are inside a "Client Pod", then run these commands: "**`nslookup web-service`**"
3. Output:

```
Name:      web-service
Address 1: 10.244.1.4  (Pod 1)
Address 2: 10.244.1.5  (Pod 2)
Address 3: 10.244.1.6  (Pod 3)
```

Action	Standard Service	Headless Service
Client Command	curl http://web-service	nslookup web-service
Result	Connects to 1 random Pod.	Gets a list of **all** Pod IPs.
Who decides?	Kubernetes decides which Pod you get.	**The Client** decides which Pod to talk to.

Doubt: If we are using a standard service, then also we can do nslookup, what is the need to make CLusterIP: none?
Ans: When you do an `nslookup` on a Standard Service, Kubernetes gives you **one single Virtual IP (the ClusterIP)**.

- **DNS Result:** `10.96.0.10`

When you set `clusterIP: None`, you are telling Kubernetes: "Don't give me a Virtual IP. Just give me the real IPs of the Pods."

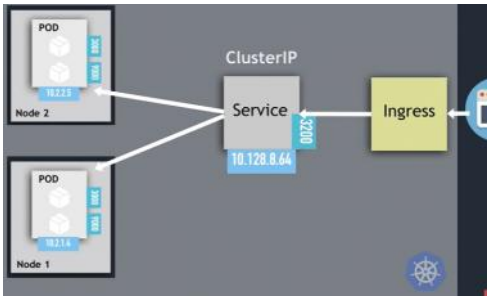
- **DNS Result:** *
  `10.244.1.5`
  `10.244.1.6`
  `10.244.1.7`

Four major Services types:

Kubernetes Services: Comparison Table

Feature	ClusterIP	NodePort	LoadBalancer	Headless (clusterIP: None)	ExternalName
Visibility	**Internal only**. Only other Pods in the cluster can see it.	**External**. Accessible via the IP of any worker node.	**External**. Accessible via a unique cloud-provided IP.	**Internal**. Used for direct Pod-to-Pod discovery.	**Outgoing**. Points from inside to *outside*.
Address	A single stable Virtual IP (VIP) and DNS name.	NodeIP:NodePort (Port range: 30000-32767).	A dedicated Public IP address (e.g., 1.2.3.4).	No Service IP. Returns a list of individual **Pod IPs**.	A **DNS Alias** (CNAME).
Analogy	**Internal Intercom**. You dial an extension to reach a department.	**A Service Window**. A hole in the building wall for outsiders.	**A Front Gate Receptionist**. A formal entrance for the public.	**A Directory Board**. A list on the wall showing everyone's room number.	**Speed Dial**. A shortcut to an outside number.
Security	**Highest**. Hidden from the internet; only accessible inside the cluster.	**Lowest**. Opens a port on every server. Attackers can see your server IPs.	**High**. Traffic is filtered by the cloud provider's firewall/LB.	**Medium/High**. Internal only, but exposes individual Pod IPs to the client.	**N/A**. Depends on the external target's security.
Best For...	Internal microservices, databases, and sidecars.	Quick testing, demos, or non-cloud environments.	**Production traffic** on AWS, Azure, or Google Cloud.	**Stateful apps** like Kafka, MongoDB, or Elasticsearch.	**Mapping external DBs** (like RDS) to a local name.
Traffic Flow	ClusterIP >>> Pod.	NodePort >>> ClusterIP >>> Pod.	LB IP >>> NodePort >>> ClusterIP >>> Pod.	DNS Result >>> Direct to Pod IP.	App >>> DNS Alias >>> **Outside URL**.
Load Balancing	**Kube-Proxy** (iptables/IPVS). Handles traffic inside the cluster.	**Kube-Proxy**. Handles traffic once it hits a Node.	**External Cloud LB** (AWS/GCP) + Kube-Proxy.	**None**. The Client App must choose which Pod to talk to.	**None**. Traffic goes straight to the external URL.

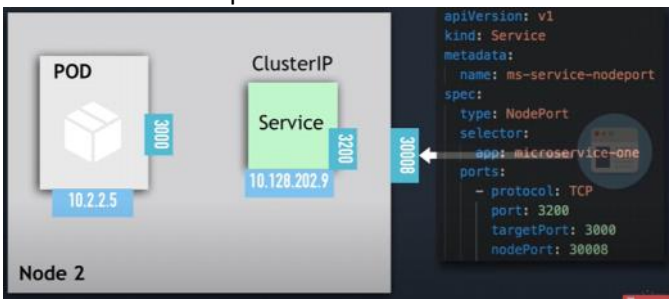
1) **ClusterIP -** is the default service type. It provides a stable IP address that is only accessible **inside** the cluster.
   a. Exposes a set of pods to other services within the same cluster. It enables communication between different services internally in the cluster
   b. The backend pods(database servers) and frontend pods(nginx web servers) in a cluster can communicate with each other via this ClusterIP service.



c. Every node is assigned with some ranges of IP addresses(Internal IP's) for ClusterIP. Such that every pod service's inside node gets an IP address called ClusterIP.
d. It creates a service inside the Kubernetes cluster, which can be accessed by other applications in the cluster, without allowing external access
e. Exposes service on a strictly cluster-Internal IP

```
apiVersion: v1
kind: Service
metadata:
  name: internal-service
spec:
  type: ClusterIP      # Default type
  selector:
    app: my-app        # Connects to pods with this label
  ports:
    - protocol: TCP
      port: 80         # Port on the Service
      targetPort: 8080 # Port on the Pod
```

2) **NodePort** - **NodePort** is built on top of ClusterIP. It opens a specific port on **every server (Node)** in your cluster so that outside traffic can get in.
A NodePort service opens a specific port on all the Nodes in the cluster, and external traffic sent to that port is forwarded to the service.



a. Service is exposed on each node's IP on a statically defined port (30000 to 32767).

Limitations:

a. It is for the external traffic, but it is not a best way for it as it opens a port to internet hence anyone can access to inside.
b. Limited ports only few thousands.

Execution flow**: Node >>> ClusterIP >>> Pod**.

Realtime example:

Imagine you have:

- **Node A** (IP: 1.1.1.1) — Running your App Pod.
- **Node B** (IP: 1.1.1.2) — **Empty** (No App Pod).
- **Service:** NodePort 31000.

What happens?

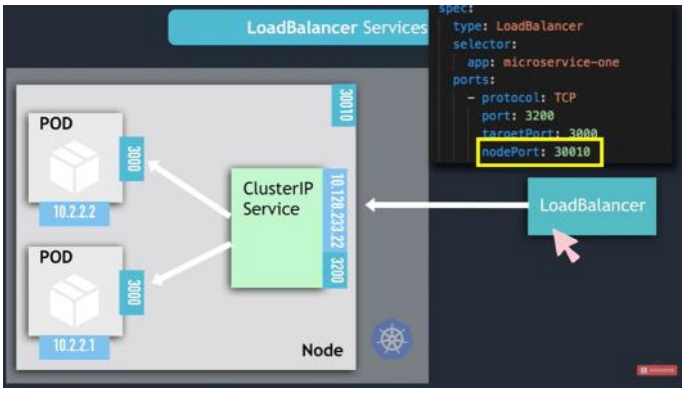
- If you go to 1.1.1.1:31000 >>>> You reach your app.
- If you go to 1.1.1.2:31000 >>>> **You still reach your app.** Node B just acts as a "relay racer" and passes the ball to Node A.

Why does Kubernetes do this?

- It makes your life easier. You don't have to track which server is currently hosting your Pods. As long as you know the IP of **any** server in the cluster, you can get to your app.

LoadBalancer - This service to expose a service outside the cluster on a static external IP.

a. Best alternative for NodePort.
b. When we create a loadbalancer service then automatically NodePort and ClusterIP service are created automatically.
c. In a GKE, this creates a Network Load Balancer with one IP address, which external users can access and are then forwarded to the relevant node in your Kubernetes cluster.



Execution flow: **LB IP >>> Node >>> ClusterIP >>> Pod**

The Execution Flow

1) **External LB IP:** The user hits the public IP (e.g., `1.2.3.4`) on Port 80.
2) **NodePort:** The Cloud Load Balancer forwards that traffic to **any** of your Cluster Nodes on a high port (e.g., `31000`).
3) **ClusterIP:** Inside the node, the networking rules (iptables/IPVS) catch that traffic and send it to the internal **ClusterIP**.
4) **App (Pod):** The ClusterIP finally picks one healthy **Pod** and sends the traffic to its internal IP.

Doubt: When we use load balancer service do we need define 3 yamls, (1 - lb, 2 - nodeport, 3 -

clusterIP)

When you apply a single `type: LoadBalancer` YAML, Kubernetes automatically does the following "behind the scenes":

1) **Creates a ClusterIP:** So the app can be reached internally.
2) **Allocates a NodePort:** It picks a random high port (e.g., 31452) and opens it on all nodes so the Load Balancer has a place to send traffic.
3) **Configures the Cloud LB:** It tells AWS/GCP to send traffic to that specific NodePort.

1) ExternalName:

- Unlike the other three services we discussed (ClusterIP, NodePort, LoadBalancer), this service **does not** use selectors and it **does not** point to any Pods. Instead, it acts as a **DNS redirect (alias)** to something sitting outside your cluster.

Purpose: An ExternalName service in Kubernetes helps you connect to services that are outside of your Kubernetes cluster.
How it works: Instead of pointing to an IP address inside your cluster, it points to a DNS name (like a website address) outside your cluster.

Example:

```
apiVersion: v1
kind: Service
metadata:
  name: database-service    # The name your App uses
spec:
  type: ExternalName
  externalName: production-db.cluster-c123.us-east-1.rds.amazonaws.com
```

How the App uses it: Your application code simply connects to http://database-service. Kubernetes handles the "translation" to the long AWS address automatically.

Why:

1. The "Single Source of Truth" (Configuration)

Imagine you have 50 different microservices that all need to talk to the same external Oracle database.

- **The Hardcoded Way:** You put oracle-db-72.aws.com in 50 different config files. If the database moves or the URL changes, you have to find and update 50 apps and restart them.
- **The ExternalName Way:** All 50 apps simply point to my-db. You only update **one** Kubernetes YAML file. The apps don't even need to restart; they just get the new "address" the next time they ask DNS.

K8s Ingress:

- While a **Service** (like NodePort or LoadBalancer) helps you reach a specific group of pods, **Ingress** is much smarter. It acts as a single entry point that manages external access to multiple services, typically using HTTP and HTTPS.

1. Why do we need Ingress?

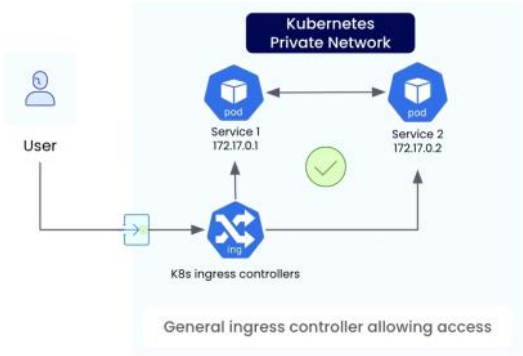
- Without Ingress, if you had 10 different apps, you might need 10 expensive Cloud Load Balancers. With Ingress, you only need **one** Load Balancer that routes traffic to the correct service based on the URL or path.
- When you have 2 different apps to expose to internet, go for ingress.
- **SSL/TLS Termination:** Handle the security certificates at the door, so your individual apps don't have to.

Kubernetes Networking Notes: Services vs. Ingress

- **Pod & Service Layer:** A Kubernetes Service exposes an application running across a set of pods. These pods can be scattered across different worker nodes (e.g., Node A runs pods for App 1, while Node B runs pods for App 2). The Service abstracts this node-level complexity and provides a stable internal network endpoint.
- **Ingress Layer (Client-Facing):** To handle external client requests, a **Kubernetes Ingress Controller** combined with an **Ingress Object** acts as a reverse proxy web server. It intercepts incoming HTTP/HTTPS traffic and routes it to the appropriate backend Service based on predefined paths or hostnames.

- **The Cost Benefit of Ingress vs. Cloud Load Balancers:**
  - If you expose applications using standard cloud load balancers (Type: LoadBalancer) directly, every single application requires its own dedicated instance. Running **10 applications would mean paying for 10 cloud load balancers**, which quickly becomes highly expensive.
  - By using an **Ingress**, you only need **one single cloud load balancer** as the entry point. The Ingress Controller handles the reverse proxy logic internally, consolidating all 10 applications behind a single external IP and massively reducing cloud infrastructure costs.

2. The Two Parts of Ingress



A. The Ingress Resource (The "Rulebook") - same like reverse proxy

- This is a YAML file where you define your routing rules. It doesn't actually *do* anything on its own; it's just a set of instructions.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: main-ingress
  namespace: dev
spec:
  ingressClassName: nginx  # Telling K8s which controller to
use
  rules:
  - host: my-company.com
    http:
      paths:
      - path: /billing
        pathType: Prefix
        backend:
          service:
            name: billing-service
            port:
              number: 80
      - path: /users
        pathType: Prefix
        backend:
          service:
            name: user-service
            port:
              number: 80
```

B. The Ingress Controller (The "Worker") - same like webserver

- An Ingress by itself is just a dumb configuration file (a YAML script) that says, *"If traffic comes to **/api,** send it to the backend service."* For that configuration file to actually do anything, you need a smart routing engine running in the cluster to read those rules and handle the network traffic. That engine is called the **Ingress Controller**.

Troubleshooting:

How to Debug Ingress

If your URL isn't working, follow this checklist:

- **Check the Controller:** Is the NGINX/Traefik pod actually running?
  - `kubectl get pods -n ingress-nginx`
- **Describe the Ingress:** Does it show an IP address under "Address"?
  - `kubectl describe ingress <name>`
- **Check the Backend:** Does the Service name and Port in your Ingress YAML match your actual Service?
- **Check the Logs:** Look at the Ingress Controller logs to see if it's rejecting the traffic.

When to avoid ingress:

- **Non-HTTP traffic:** Your apps use a protocol that isn't HTTP/HTTPS (like specialized gaming protocols or raw TCP).
- **Extreme Isolation:** You have a strict requirement that App A and App B must never share the same entry point for security or compliance reasons.

Loadbalancer Service vs Ingress

a.

Feature	LoadBalancer Service	Ingress
Number of Load Balancers	One per application	One for the whole cluster
Routing Logic	IP and Port only	URL Paths and Hostnames
Cost	High ($$ for every app)	Low ($ for one entry point)
Management	Simple, but doesn't scale	Requires an Ingress Controller
Traffic Layer	Layer 4	Layer 7
Speed	Fast	Slighly slow

What is layers define here:

- **Layer 4 (Transport):** This is like the **Address on the Envelope**. It only looks at the "To" and "From" addresses (IP addresses) and the "Door Number" (Port). It doesn't care what is inside the box.
  - **Data sees** - IP Address & Port,
  - Super Fast

  When to use it:
  - **Non-HTTP protocols:** Databases (PostgreSQL on port 5432, MySQL on 3306), Redis (6379), Kafka brokers, MQTT (IoT devices).

  How it works:
  - "Take any traffic coming into Port 5432 and forward it directly to Pod X."

- **Layer 7 (Application):** This is like **Opening the Letter** and reading the contents. It looks at the actual data—the words, the images, or the specific requests written inside.
  - **Data sees** - URL, Cookies, HTTP Headers,
  - Slightly slower (it need to open and see data)

  When to use it:
  - Web applications, REST APIs, GraphQL servers, microservice-to-microservice web calls.

  How it works:
  - "Inspect the HTTP request. If the path is /api/users, send it to Service A. If the HTTP Header

contains User-Agent: Mobile, send it to Service B."

Layer 7 Advanced features:

a. **HTTP Path & Header Matching (smart routing):** Instead of routing all traffic to one place.
   - **Path-based:** facebook.com/feed goes to the **Feed Service**, while facebook.com/settings goes to the **User Service**.
   - **Header-based (A/B Testing & Canary Deployments):** If the request header includes x-beta-user: true, route that user to the **New V2 App Deployment**. Everyone else stays on **V1**.

b. **Retries (Automated Resilience)**
   - If Microservice A calls Microservice B and Microservice B drops the connection or returns a 503 Service Unavailable due to a temporary network glitch:
     - **Without L7 Retries:** The end user immediately sees an error screen (500 Server Error).
     - **With L7 Retries:** The proxy automatically intercepts the 503 response behind the scenes and silently retries the request 2 or 3 times in a split second. The user never notices anything went wrong.

c. **Rate Limiting (DDoS & Abuse Protection)**
   - Controls how many requests a client or service can make in a given timeframe:
     - **Example:** You restrict an API endpoint so that a single IP address can only make **100 requests per minute**. If they exceed 100, the proxy instantly returns 429 Too Many Requests without letting the spam traffic hit or overload your backend database.

- **Simple Namespace (Needs L4 Only):** Imagine a namespace called analytics-pipeline. It only contains background workers processing raw binary data streams from Kafka to PostgreSQL.
  - It **does not need** HTTP header routing, rate limiting, or path matching.
  - **Result:** It uses **ztunnel** (L4) for fast mTLS encryption. It saves money and CPU because it **does not need** an Envoy/Waypoint proxy.
- **Complex Namespace (Needs L7):** Imagine a namespace called checkout-system. It hosts APIs that handle user credit cards, discount codes, and user sessions over HTTP/gRPC.
  - It needs **rate limiting** (to stop bots), **retries** (so payment calls don't fail on transient drops), and **header routing** (to route mobile users differently).
  - **Result:** You deploy a **Waypoint Proxy** for this namespace so Envoy can perform those L7 functions.

1. Cloud Load Balancer vs. Ingress Controller

Why put a Cloud Load Balancer in front of an Ingress Controller?

- **Physical Network Boundary:** Ingress Controller pods (NGINX, Envoy) run on private node networks inside the cluster and do **not** have public IP addresses.
- **Public Entry Point:** The Cloud Load Balancer (AWS ALB, GCP HTTP LB) lives on the cloud provider's edge network, providing a static **Public IP** accessible from the internet.

End-to-End Traffic Flow

$$Internet \xrightarrow{Public\ IP} Cloud\ Load\ Balancer \xrightarrow{NodePort\ /\ Private\ IP} Ingress\ Controller\ Pod \xrightarrow{Path\ Routing} App\ Pods$$

1. **Cloud Load Balancer:** Receives external traffic on Ports 80/443 via its Public IP and routes it into the cluster.

2. **Ingress Controller:** Inspects HTTP host/path headers (e.g., /api or /users) and routes the request to the appropriate backend application pods.

2. Ingress Architecture vs. Direct Type: LoadBalancer

While both mechanisms work, they represent two distinct architectural patterns. Combining both directly for a single application is usually redundant and unnecessary.

Pattern A: Ingress Architecture (Enterprise Standard)

Cloud Load Balancer ⟶ Ingress Controller ⟶ ClusterIP Service ⟶ App Pods

- **How it works:** A single Cloud Load Balancer and Ingress Controller sit at the cluster edge. Traffic is routed internally using lightweight ClusterIP services.
- **Best for:** Web applications and HTTP/HTTPS microservices.
- **Key Advantage: Cost-efficient.** Manages dozens of microservices behind a single paid Cloud Load Balancer using path-based or host-based routing.

Pattern B: Direct Type: LoadBalancer Service

Cloud Load Balancer ⟶ LoadBalancer Service ⟶ App Pods

- **How it works:** Setting $type: LoadBalancer$ triggers the cloud provider to dynamically spin up a dedicated Cloud Load Balancer directly targeting that specific service's pods.
- **Best for:** Non-HTTP workloads (raw TCP/UDP, databases) or isolated, simple deployments without complex path routing needs.
- **Trade-off: Expensive at scale.** Every service creates its own Cloud Load Balancer, leading to higher cloud costs.

3. How Ingress Controllers Are Exposed

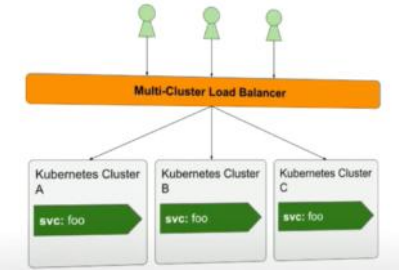
Ingress Controllers actually use $type: LoadBalancer$ behind the scenes to bootstrap themselves:

1. **Ingress Controller Pods:** The controller software reading routing rules.
2. **Ingress Service (type: LoadBalancer):** The service attached to those Ingress Pods that tells the cloud provider to build the single, entry-point Cloud Load Balancer.

Your backend applications then only need simple, cost-free internal **ClusterIP Services**.

Multi Cluster Ingress :

Multi-Cluster Ingress (MCI) is a cloud-native networking architecture and API resource designed to route external internet traffic (North/South traffic) across applications deployed across **multiple Kubernetes clusters**, often spanning different geographic regions.



Feature:

- High **availability** or no downtime
- Disaster **recovery** like backup and restore
- **Scalability** or high performance.

Helm Charts

1. Definition

Helm is the package manager for Kubernetes. If Kubernetes is the Operating

System, **Helm** is the "App Store" or yum/apt equivalent. A **Helm Chart** is a collection of files that describe a related set of Kubernetes resources (like Deployments, Services, and Ingress) bundled together as a single unit.

2. Analogy: The "IKEA Furniture" Set

Imagine you are building a house:

- **Kubernetes YAML:** Is like buying raw wood, nails, and glue. You have to measure and build every piece of the chair manually. If you want a second chair, you have to do all the measurements again.
- **Helm Chart:** Is like an **IKEA Flat-Pack**. All the pieces (YAML files) are pre-cut and ready. It comes with an **Instruction Manual** (Values file).
    - If you want a "Red Chair," you just change the "Color" setting in the manual.
    - If you want "4 Chairs," you just change the "Quantity" setting.
    - Helm does the hard work of "assembling" it in your cluster.

3. More Description (The "Why")

Standard Kubernetes YAML files are **static**. If you have 10 different environments (Dev, QA, Prod), you would normally need 10 different sets of YAMLs. This is hard to maintain.

- Helm solves this by using **Templating**. It takes a standard YAML file and replaces specific values with "placeholders" (variables). When you run Helm, it injects your specific settings into those placeholders to create the final manifest.
- Ex: If we want some other service into our cluster like "Elastic stack for logging" we need to add a Statefulset, configMap, secret, user with permissions and services YAML files into the cluster, hence it is burden so those yaml configurations are there in a package and be used those bundle of YAML files are called "Helm Charts".

4. The Three Core Components

- **Chart.yaml:** The "Metadata" file. It contains the name of the chart, the version, and a description. meta info about chart
- **values.yaml:** The "Configuration" file. This is where you store your variables (e.g., replicaCount: 3, imageTag: "v1.2.0"). This is the only file a user usually needs to edit.
- **templates/ folder:** The "Blueprints." This contains the actual Kubernetes YAML files with placeholders like {{ .Values.replicaCount }} instead of hardcoded numbers.

```
my-chart/
 ├── Chart.yaml                  # Metadata about the chart (version,
description, etc.)
 ├── values.yaml                 # The default configuration values for the
templates
 ├── charts/                     # A directory containing any sub-charts
(dependencies)
 ├── templates/                  # The directory containing Kubernetes
manifest blueprints
 │   ├── NOTES.txt               # Plain text file containing usage
instructions displayed after install
 │   ├── _helpers.tpl            # Named templates/snippets used across the
chart files
```

```
│   ├── deployment.yaml         # Blueprint for a Kubernetes Deployment
│   ├── hpa.yaml                # Blueprint for a HorizontalPodAutoscaler
│   ├── ingress.yaml            # Blueprint for a Kubernetes Ingress resource
│   ├── service.yaml            # Blueprint for a Kubernetes Service
│   ├── serviceaccount.yaml     # Blueprint for a Kubernetes ServiceAccount
│   └── tests/          # Integration tests for your chart
│       └── test-connection.yaml
└── .helmignore         # Specifies files to ignore when packaging the chart
```

5. Simple Commands (The Workflow)

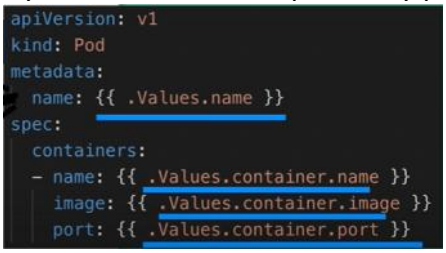
Action	Command	Description
Search	helm search hub [app]	Find a pre-made chart (like Nginx or Grafana) from the community.
Install	helm install [name] [chart]	Deploy the app to your cluster.
Upgrade	helm upgrade [name] [chart]	Change settings or update the app version without downtime.
Rollback	helm rollback [name] [rev]	If the update fails, instantly go back to the previous working version.
Uninstall	helm uninstall [name]	Remove all associated resources (Pod, Service, etc.) with one command.

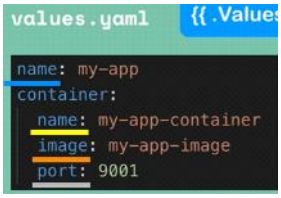
6. Example: One Chart, Many Flavors

Instead of having a prod-deployment.yaml and a dev-deployment.yaml, you have one **AlloyDB-App Chart**.

- **For Dev:** You run Helm with --set replicaCount=1.
- **For Prod:** You run Helm with --set replicaCount=3 --set resources.limits.cpu=1000m.

- Can create your own helm charts with helm.
- Push them to helm repository.
- Download and use existing ones.
- Second feature of Helm is, it is a templating engine.
    - Define a common blueprint.
    - Dynamic values are replaced by placeholders.





- Tiller observed it has too much powers hence it was removed from Helm 3.

Realtime Example of Helm charts:

1. The Scenario: "The Monitoring Request"

Imagine your manager asks you to set up full monitoring for a new Kubernetes cluster.

- **The Manual Way (No Helm):** You would have to find, write, and manage about **15 to 20 different YAML files** (Deployments for Prometheus, ConfigMaps for Grafana dashboards, Services for networking, RBAC Roles for permissions, and PersistentVolumeClaims for data). If you make a typo in one file, the whole system might fail.
- **The Helm Way:** You use the **kube-prometheus-stack** Helm Chart. This is a pre-packaged "bundle" created by the community that includes everything you need.

2. The Real-Time Workflow

Step A: Adding the Repository

First, you tell Helm where the "App Store" for monitoring is located.
Bash

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
```

Step B: Customizing via values.yaml

You don't want the "default" setup. You want Grafana to have a specific admin password and you want to enable persistence so you don't lose data if a pod restarts. You create a small my-values.yaml file:
YAML

```
grafana:
  adminPassword: "SecurePassword123"
  persistence:
    enabled: true
    size: 10Gi
prometheus:
  prometheusSpec:
    retention: 30d
```

Step C: The "One-Command" Deployment

Now, instead of applying 20 files, you run one command:
Bash

```
helm install monitoring-stack prometheus-community/kube-prometheus-stack -f my-values.yaml
```

3. Why this is a "Real-Time" lifesaver

Situation	Without Helm	With Helm
Updating	You have to manually edit 5 different YAMLs to change an	Run helm upgrade with the new version flag.

	image version.	
Scaling	You hunt through a 200-line Deployment file for replicas.	Change replicaCount in your values.yaml.
Disaster Recovery	You hope you saved all 20 YAMLs in Git correctly.	Run helm install using your saved values.yaml.
Rollback	If a change breaks the site, you have to manually undo your YAML edits.	Run helm rollback monitoring-stack 1 to instantly return to the previous version.

4. Comparison to your Experience

If you were deploying **AlloyDB Omni** in Kubernetes:

- **The Chart:** Would contain the blueprint for the Postgres engine, the Patroni agent, and the etcd sidecars.
- **The Values:** You would just swap out things like databaseName: sales_db or storageSize: 100Gi.

Important Helm chart Topic:

Does helm uninstall delete everything?

⚠ **This is where you need to be careful.**

Don't think:

"helm uninstall wipes everything associated with the application."

It removes resources that Helm is managing as part of that release, but there are important exceptions and behaviors depending on how resources were created and annotated.

For example, a resource can have Helm's keep policy:

```
metadata:
annotations:
helm.sh/resource-policy: keep
```

Such a resource is intentionally retained when the release is uninstalled.

So:

```
Helm-managed resource
|
├── normal resource → usually deleted
|
└── resource-policy: keep → retained
```

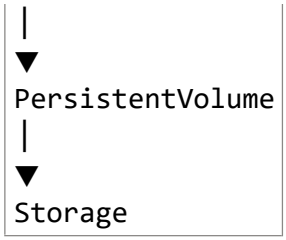
This is particularly important for resources containing persistent data.

5. What about PersistentVolumes / PVCs? 💾

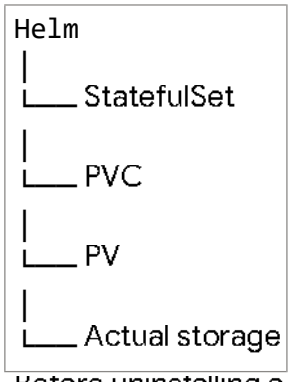
This is an important Kubernetes + Helm scenario.

Suppose your chart creates:

```
StatefulSet
|
▼
PVC
```



If you uninstall the Helm release, **don't assume your data is automatically safe.**
The behavior depends on how the storage resources were created and the Kubernetes storage configuration, including the PVC/PV lifecycle and reclaim policy.
For example:



Before uninstalling a production stateful application, you should explicitly check:

kubectl get pvc -n production
and:

kubectl get pv
This is a very good operational habit.

Doubt:

1. **That means helm uninstall will delete storage pvc as well?**

   Not necessarily. Whether helm uninstall deletes a PersistentVolumeClaim (PVC) and its underlying storage depends on **how the PVC was created** and the **PersistentVolume (PV) Reclaim Policy.**

   ## 1. PersistentVolumeClaim (PVC) Deletion Behavior

   - **If the PVC is explicitly defined in the Chart templates (templates/pvc.yaml):**
     - Helm **will delete** the PVC resource object from Kubernetes when you run helm uninstall.
   - **If the PVC was created dynamically by a volumeClaimTemplates (e.g., inside a StatefulSet):**
     - Kubernetes **does NOT delete** PVCs created by StatefulSet volume claim templates upon release/workload deletion. Running helm uninstall will remove the StatefulSet, but the PVCs will **remain intact** in the cluster.
   - **If the PVC is managed outside the Chart:**
     - Helm will not touch it.

2. What happens to the actual Data/Storage (PV)?

Even if Helm deletes the PVC, the underlying disk/storage (the **PV**) depends on the StorageClass **persistentVolumeReclaimPolicy**:

- **Delete (Default for most cloud providers):** When the PVC is deleted, the Kubernetes control plane automatically **deletes the underlying cloud storage disk/volume**, destroying the data.
- **Retain:** When the PVC is deleted, the PV and physical storage remain intact, retaining all data so an administrator can manually recover or reuse it.

Summary Checklist Before Uninstalling

a. **Check if PVCs exist for the release:**
   Bash

   ```
   kubectl get pvc -n <namespace>
   ```

b. **Check the Reclaim Policy of your storage:**
   Bash

   ```
   kubectl get pv
   ```

2. **If the PVC was created dynamically by a volumeClaimTemplates (e.g., inside a StatefulSet): in this case, pvc becomes orphan?**

Yes, exactly. In Kubernetes terminology, those PVCs become **orphaned** (or "unmanaged") resources.

What "Orphaned" Means in This Context

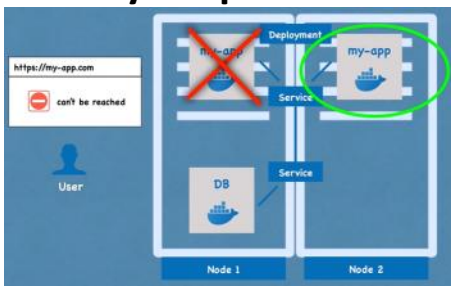
- **Unbound from the workload:** When Helm uninstalls the StatefulSet, the pods and the StatefulSet object are deleted, but the PVCs remain sitting in your namespace.
- **Not tracked by Helm:** Because the PVCs were dynamically generated by the StatefulSet's volumeClaimTemplates (rather than being explicitly defined as standalone resources in Helm's templates/ directory), Helm never direct-managed them. Thus, helm uninstall leaves them completely untouched.
- **Data Preserved:** The PVC stays bound to its PersistentVolume (PV), meaning your underlying data remains intact.

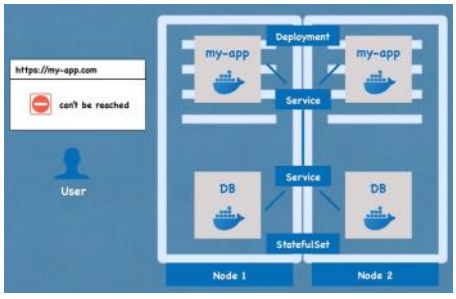
What Happens Next?

- **If you reinstall/upgrade the chart:** If you deploy a StatefulSet with the same name and template, Kubernetes will automatically reattach the existing PVCs (e.g., data-my-statefulset-0) to the new pods, preserving your previous state.
- **If you want to clean them up permanently:** You must manually delete the orphaned PVCs using kubectl:
  Bash

  ```
  kubectl delete pvc -l app.kubernetes.io/instance=<release-name> -n <namespace>
  ```

When your pod is out of service then ?????





Deployments and replica sets:

- **Deployment (Kind: Deployment):** The high-level **Manager**. It handles the "Life Cycle" of your app—updates, rollbacks, and deployment strategies.
- **ReplicaSet (Kind: ReplicaSet):** The mid-level **Supervisor**. Its only job is to ensure that the exact number of Pods you requested are running at all times (Self-healing).
- **Pod (Kind: Pod):** The low-level **Worker**. This is the actual container running your application code.
- Such that Deployments for stateless apps, and StatefulSets for Stateful apps or databases.

ReplicaSets:

Purpose: A ReplicaSet ensures that a specified number of pod replicas are running at any given time.

When to use:

- You need to maintain a fixed number of replicas for your application, and you don't need advanced deployment features.
- You have a simple use case where you only need to ensure the number of pod replicas, without version control or rollback capabilities.

Limitations:

- Does not support rolling updates, rollbacks, or declarative updates to Pods.

The Relationship: Deployment → ReplicaSet → Pod

You almost never create a ReplicaSet manually. Instead, when you create a **Deployment**, it automatically creates a **ReplicaSet** for you.

Why do we need the ReplicaSet in the middle?

The ReplicaSet's only job is **Self-Healing**. It constantly monitors the cluster to ensure that the number of running Pods matches the replicas number you defined.

- If a Pod crashes, the ReplicaSet notices and starts a new one.
- If a Node dies, the ReplicaSet notices the Pods are gone and starts them on a healthy

Node.

Key Differences: Why we use Deployments

Even though ReplicaSet is a valid resource kind, we use Deployment because it provides "Brains" that the ReplicaSet lacks

Feature	ReplicaSet	Deployment
Primary Goal	**Quantity:** "Are there 3 pods?"	**Quality & Strategy:** "Are there 3 pods of Version 2?"
Updates	**Manual:** You must delete pods to see new changes.	**Automated:** Automatically rolls out new versions.
Rollbacks	No history; no "undo" command.	Keeps history; supports rollout undo.
Downtime	Often requires downtime for updates.	Supports Zero-Downtime Rolling Updates.

Troubleshooting:

For your interview, avoid shortcuts. Use the full resource names to show deep understanding of the API.

View current status:	`kubectl get deployments,replicasets,pods`

Check the progress of an update:	`kubectl rollout status deployment [deployment-name]`
	
	`Or`
	
	`kubectl get deployment my-app -o jsonpath='{.status.conditions}'`

View the history of versions	`kubectl rollout history deployment [deployment-name]`
Emergency Rollback to previous version	`kubectl rollout undo deployment [deployment-name]`
Troubleshoot a failure (Events & Logs)	`kubectl describe deployment [deployment-name]`

Version Update Process:

1. **Modify the File:** You open your **deployment.yaml** and change image: **my-app:v1** to image: **my-app:v2**.
2. **Commit:** You save this change in Git. This creates a permanent record of *who* changed the version and *when*.
3. **Apply:** You run: `kubectl apply –f deployment.yaml`
   a. **Why apply not edit? -> Refer sample interview questions (6).**

What happens inside the Cluster? (The Step-by-Step)

When you run that command, a "choreography" begins inside the brain of Kubernetes:

a. **The API Server** receives your V2 YAML and updates the record in **etcd**.
b. **The Deployment Controller** notices the version change. It says: *"I need to move from V1 to V2 using the RollingUpdate strategy."*

c. **New ReplicaSet:** It creates a **new ReplicaSet** for V2.
d. **Scaling Up:** The new ReplicaSet starts 1 Pod with the V2 image.
e. **Health Check:** Kubernetes waits for that V2 Pod to pass its **Readiness Probe** (to make sure it's actually working).
f. **Scaling Down:** Once V2 is healthy, the **old ReplicaSet** (V1) terminates 1 Pod.
g. **Repeat:** This continues until all Pods are V2.

Troubleshooting:

a. Watch the Pods swapping: `kubectl get pods --watch`
b. Check the Rollout status: `kubectl rollout status deployment/web-app`
c. If update fails, undo deployment: `kubectl rollout undo deployment/web-app`

Deployment techniques:

There are several update strategies for deployment, default is **rollingUpdate**:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 10
  # --- STRATEGY SECTION START ---
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%        # How many extra pods can be created (10 + 2.5
= 13 pods total during update)
      maxUnavailable: 25%  # How many pods can be down at once (at least 7
pods always running)
  # --- STRATEGY SECTION END ---
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: nginx
        image: nginx:1.22
```

1. Rolling Update (The Default)

Kubernetes gradually replaces old Pods with new ones.

- **Process:** It spins up one new Pod, waits for it to be "Ready," then kills one old Pod.
- **Pros:** Zero downtime; no extra infrastructure cost.
- **Cons:** "Version Skew"—for a few minutes, some users see V1 and others see V2. Your database must support both versions simultaneously.
- **Deployment object:** `spec.strategy.type: RollingUpdate`

2. Recreate (The "Clean Slate")

All existing Pods are killed before any new Pods are created.

- **Process:** Replicas go 3 → 0, then 0 → 3.
- **Pros:** No version skew. Only one version of the app exists at a time. Perfect for apps that cannot handle two versions talking to the same database.
- **Cons: Downtime.** Users will see errors until the new version is fully booted.
- **Deployment object:** `spec.strategy.type: Recreate`

3. Blue-Green (The "Flip")

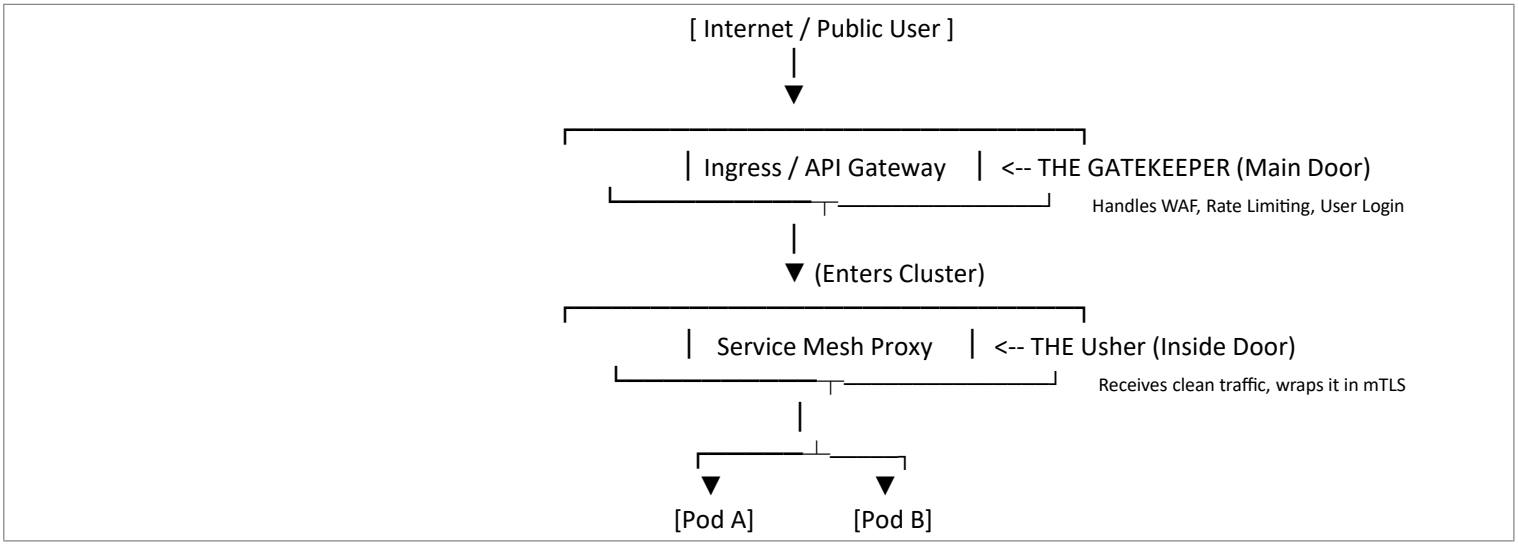
Two identical environments exist. "Blue" is live; "Green" is the new version being tested.

- **Process:** You deploy V2 to the "Green" environment. Once tested, you flip the **Service** or **Load Balancer** to point to Green.
- **Pros:** Instant cutover and **Instant Rollback** (just flip the switch back).
- **Cons: Expensive.** You are paying for 2x the servers during the deployment phase.
- **Deployment object: No, Blue-Green:** You create **two separate Deployment files** (one named app-blue, one named app-green) and you update a **Service** to point to one or the other.

4. Canary (The "Test Drive")

A tiny percentage of traffic is routed to the new version to "test the waters."

- **Process:** 95% of users stay on V1; 5% go to V2. If the 5% experience errors, you abort. If they are happy, you roll out to 100%.
- **Pros:** Lowest risk. You find bugs with real users without affecting everyone.
- **Cons:** Complex to set up. Usually requires a **Service Mesh** (like Istio) or an **Ingress Controller** that supports traffic splitting.
- **Deployment object:** NO, **Canary:** You use a **Service Mesh (Istio)** or an **Ingress Controller** to split traffic between two different Deployment objects.



- **Ingress:** Acts as the external gatekeeper (North-South proxy) that routes incoming public internet traffic to specific applications inside the cluster based on host or URL path rules.
- **Service Mesh:** Acts as the internal usher (East-West proxy system) that secures, encrypts (mTLS), and manages traffic flowing *between* microservices deep inside the cluster.

5. Shadow (The "Mirror")

V2 receives the exact same traffic as V1, but the responses from V2 are discarded.

- **Process:** Traffic is "mirrored." V1 handles the real user request. V2 processes the same request in the background to see how it performs under real load.
- **Pros:** Zero impact on users. You can test performance and logic with **real production data** safely.
- **Cons:** Expensive and complex. You must ensure V2 doesn't trigger "side effects" (like sending a duplicate email to a customer or double-charging a card).

6. A/B Testing (The "Business Test")

Similar to Canary, but traffic is split based on **User Attributes** (e.g., location, browser, or "Beta" status).

- **Process:** You send users from Hyderabad to V2 and users from Bangalore to V1.
- **Goal:** This isn't just for stability; it's to see if Version 2 increases sales or engagement.

7. Big Bang (The "All-at-Once")

The entire system is updated simultaneously across all components (App, DB, Cache).

- **Process:** Usually involves a scheduled maintenance window.
- **Pros:** Ensures all components are perfectly synced.
- **Cons:** High risk. If it fails, the "Bang" is literal—everything goes down, and rollbacks are very difficult.

StatefulSets:

1. Why ReplicaSet fails for Databases

If you use a ReplicaSet for a database (like MySQL or MongoDB), you run into two major problems:

a. **Identity:** ReplicaSets give pods random names like mysql-abc12. If that pod dies and a new one starts as mysql-xyz99, your other services won't know where to find the database.
b. **Storage:** In a ReplicaSet, all pods try to share the same disk. If 3 database pods write to the same file at the same time, your data becomes corrupted.

2. The Solution: StatefulSet (Kind: StatefulSet)

A StatefulSet is designed specifically for applications that need a **persistent identity** and **unique storage**.

The Three Pillars of StatefulSets:

A. Deterministic Naming (Ordinal Index)

Instead of random strings, pods are named starting from zero: mysql-0, mysql-1, mysql-2.

- If mysql-0 dies, it is replaced by a new pod **with the exact same name**.
- This allows other services to always know the "address" of the database.

B. Stable Network Identity (Headless Service)

StatefulSets use a "Headless Service" to give each pod its own DNS record. You can talk specifically to the leader: mysql-0.database.svc.cluster.local.

C. Dedicated Storage (VolumeClaimTemplates)

This is the "magic" of StatefulSets. Each pod gets its **own private disk** that follows it.

- mysql-0 gets disk-0.
- mysql-1 gets disk-1.
- If mysql-0 is moved to a different server (Node), Kubernetes unplugs disk-0 from the old server and plugs it into the new one. The data is never lost.

3. Side-by-Side Comparison

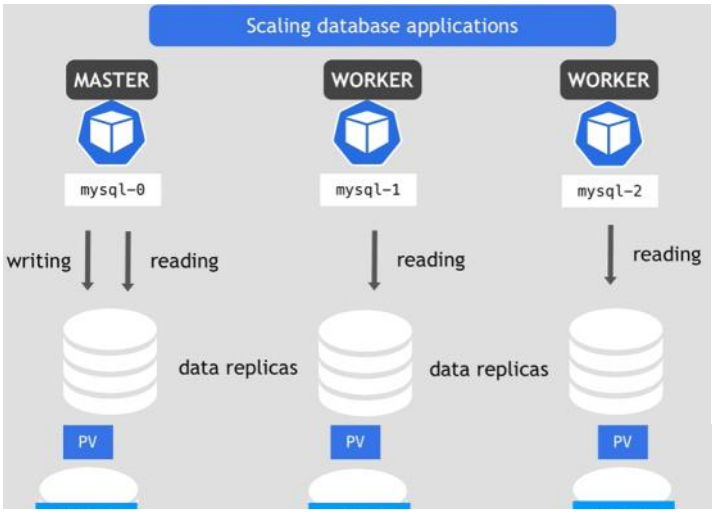
Feature	Deployment / ReplicaSet (Stateless)	StatefulSet (Stateful/DB)
Pod Names	Random (e.g., web-7gh2x)	Ordered (e.g., db-0, db-1)
Storage	Shared or Temporary	**Dedicated per Pod**
Startup Order	All at once (Parallel)	One by one (0, then 1, then 2)
Best For	Nginx, Python APIs, Microservices	MySQL, Postgres, Kafka, MongoDB

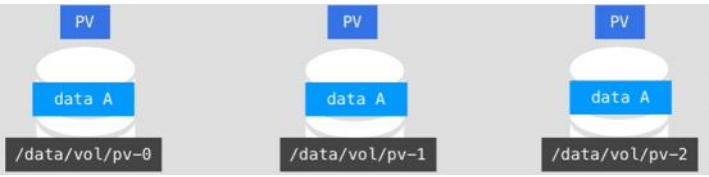
4. Example Code (StatefulSet)

Notice the volumeClaimTemplates section—this is how you tell Kubernetes to give every pod its own 10GB disk.

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: "mysql-service" # Needs a headless service
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
  # --- THIS IS THE KEY PART ---
  volumeClaimTemplates:
  - metadata:
      name: data
    spec:
      accessModes: [ "ReadWriteOnce" ]
      resources:
        requests:
          storage: 10Gi
```

- A StatefulSet maintains a **sticky identity** for each of their Pods. They are created from same specification, but not interchangeable.
  - Persistent identifier across any re-scheduling.
  - Sticky Password recognizes when you are registering on a website and offers you to save your personal details as an **Identity**, which you can use to quickly fill out online forms.





- They do not use the same physical storage.
- Continuously synchronise the data, and worker must know about each change to be up-to-date.
- If a worker3 is created then it needs to clone from the worker2 database and needs to be continuously synchronized by itself.

- **Need of StatefulSets** :It needs persistent storage so that the hosted application saves its state and data across restarts.
  - In the above mechanism data will survive, even when all pods die. As persistent volume lifecycle isn't tied to other component's lifecycle.

DaemonSet (Kind: DaemonSet)

A **DaemonSet** ensures that **all (or some) Nodes run a copy of a Pod**. As nodes are added to the cluster, Pods are automatically added to them.

- **The Logic:** You don't specify a "replica count." The count is automatically equal to the number of nodes. **No** node will have more than one pod.
- **Why not use a Deployment?** If you have 10 nodes and a Deployment with 10 replicas, Kubernetes might put 2 pods on one node and 0 on another. A DaemonSet guarantees a **1-to-1 relationship** per node.

- **Question:** What happens to a DaemonSet when a new node is added to a Kubernetes cluster, and does it require manual intervention?
  - **Answer:** No manual intervention is needed because the DaemonSet controller automatically detects the new node during continuous reconciliation and schedules a Pod on it to match the desired state.

  - What is D

- **Common Uses:**
  - **Log Collectors:** (e.g., Fluentd or Logstash) to collect logs from every server.
  - **Monitoring Agents:** (e.g., Prometheus Node Exporter) to check the health of every server.
  - **Networking:** (e.g., Calico or kube-proxy) to manage traffic on every server.

```yaml
# 1. API & KIND
apiVersion: apps/v1
kind: DaemonSet        # <--- Specific Kind for "One Pod Per Node"
metadata:
  name: fluentd-log-collector
  namespace: kube-system # Usually infrastructure tools live here
  labels:
    k8s-app: fluentd-logging
# 2. SPEC (The Controller Logic)
spec:
  selector:
    matchLabels:
      name: fluentd-pod
```

```
  # 3. TEMPLATE (The Pod Blueprint)
  template:
    metadata:
      labels:
        name: fluentd-pod
    spec:
      # TOLERATIONS: This is a Senior-level field!
      # It ensures the pod can run on the Master/Control-Plane nodes too.
      tolerations:
      - key: node-role.kubernetes.io/master
        effect: NoSchedule

      containers:
      - name: fluentd
        image: fluentd:v1.14
        resources:
          limits:
            memory: 200Mi
          requests:
            cpu: 100m
            memory: 200Mi

        # DaemonSets often need to look at the Node's files (like logs)
        volumeMounts:
        - name: varlog
          mountPath: /var/log

      # 4. VOLUMES
      # HostPath is common in DaemonSets because they interact with the
physical server
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
```

Since you've mastered how Kubernetes manages **Stateless** apps (Deployments) and **Stateful** apps (StatefulSets), the natural "next level" for a senior professional is understanding **DaemonSets** and **Jobs**. While a Deployment says *"Give me 3 pods anywhere,"* these resources say *"I need pods in a specific pattern."*

Jobs & CronJobs (Kind: Job / CronJob)

Everything we've discussed so far (Deployments, StatefulSets) is meant to run **forever**. But what if you just want to run a script and then stop?

A. Job

A **Job** creates one or more Pods and ensures that a specific number of them successfully **terminate (exit 0)**.

- **Usage:** Database migrations, heavy data processing, or generating a one-time report.
- **Senior Tip:** If a Job's Pod fails, the Job will keep creating new ones until the task succeeds (based on your backoffLimit).

```
apiVersion: batch/v1
kind: Job              # <--- Specific Kind for tasks that finish
metadata:
  name: db-migration-job
spec:
  # BACKOFF LIMIT: If the job fails (exit 1), K8s will retry it 4
```

```
times
  # before giving up. Default is 6.
  backoffLimit: 4

  # COMPLETIONS: If you need this task to run 3 times successfully
  # before being "Done".
  completions: 1
  template:
    spec:
      containers:
      - name: migration-tool
        image: migrate/tool:v2.1
        command: ["/bin/sh", "-c", "python manage.py migrate"]

      # RESTART POLICY: This is CRITICAL.
      # Jobs cannot use 'Always'. They must use 'OnFailure' or
'Never'.
      restartPolicy: OnFailure
```

Feature	ReplicaSet (Deployment)	Job
Desired State Goal	Keep N number of pods active forever.	Successfully complete N number of tasks.
Allowed Restart Policies	Always	OnFailure or Never
Scaling Mechanism	Horizontal scaling via replicas count.	Managed via completions (total tasks) and parallelism (simultaneous pods).
Ideal Use Case	NGINX frontend, Spring Boot API, Node.js backend.	Running a database schema migration script before a new version roll out.

B. CronJob

A **CronJob** is a Job that runs on a **time schedule** (using Linux Crontab syntax).

- **Usage:** Taking a database backup every night at 2:00 AM, or sending weekly email reports.
- **Example Schedule:** 0 2 * * * (Every day at 2 AM).

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: nightly-backup
spec:
  # SCHEDULE: Minute | Hour | Day of Month | Month | Day of Week
  # This runs at 00:00 (Midnight) every single day.
  schedule: "0 0 * * *"

  # CONCURRENCY POLICY: What if the 12:00 AM backup is still running
  # when the 1:00 AM one starts?
  # 'Forbid' prevents the new one from starting.
  concurrencyPolicy: Forbid
  # SUCCESS/FAILED HISTORY LIMIT: How many old job records to keep in
etcd.
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
```

```
          containers:
          - name: backup-worker
            image: alpine:latest
            command: ["/bin/sh", "-c", "echo 'Backing up DB...'; sleep
30"]
          restartPolicy: OnFailure
```

Kubernetes Workload Comparison Table

Resource	Purpose	Persistence	Pod Naming	Storage	Common Use Cases
Deployment	**Stateless Scaling**	Forever	Random (e.g., web-ab12)	Shared / Ephemeral	Web APIs, Frontends, Microservices
Stateful Set	**Stateful Identity**	Forever	Ordered (e.g., db-0, db-1)	**Dedicated per Pod**	Databases (MySQL), Kafka, Redis
Daemon Set	**Node Coverage**	Forever	1 per Node	Usually HostPath	Logs (Fluentd), Monitoring (Prometheus)
Job	**One-time Task**	Temporary	Random	Temporary	DB Migrations, Batch processing
CronJob	**Scheduled Task**	Temporary	Random	Temporary	Nightly Backups, Weekly Reports

Technical Spec Comparison

Resource	Restart Policy	Scaling Logic	Update Strategy	Key YAML Fields
Deployment	Always	Replica Count	RollingUpdate / Recreate	replicas, strategy
StatefulSet	Always	Replica Count (Ordered)	RollingUpdate / OnDelete	serviceName, volumeClaimTemplates
DaemonSet	Always	Auto (1 per Node)	RollingUpdate / OnDelete	tolerations, selector
Job	OnFailure / Never	completions	N/A	backoffLimit, activeDeadlineSeconds
CronJob	OnFailure / Never	Time-based	N/A	schedule, concurrencyPolicy

Custom Resources & Operators

Why StatefulSet isn't enough

A StatefulSet is a "dumb" manager. It isn't "smart" enough to handle a complex database like **Elasticsearch** or **Postgres** (which need specific logic for backups and failovers).

- If a pod in a StatefulSet crashes, it restarts it. **That's it.** It knows how to keep a Pod running.
- It doesn't know how to **upgrade** the database version without corrupting data.
- It doesn't know how to "Reindex a Database" or "Perform a Point-in-Time Recovery."
- It doesn't know how to **replicate** data from a leader to a follower.

- **The Solution:** You use an **Operator**.
- **What it is:** It is a custom piece of code running in your cluster that acts like a **human administrator**. It knows how to "operate" the application perfectly, beyond what a standard YAML file can do.
- **The Operator adds the "Smarts."** It knows: *"Before I restart this pod, I must make sure the other two nodes are healthy and have taken over the leadership."*

Analogy of operator:

- To understand an **Operator**, you first have to understand how a human **Systems Administrator** (like you) works.
- When a database breaks at 2:00 AM, a human admin logs in, checks the logs, sees which node failed, promotes a follower to a leader, and restores the backup. **An Operator is that "Human Admin" turned into software code.**
- The standard Kubernetes Control Plane (the "Brain") only knows how to speak "Basic Kubernetes." It knows what a **Pod**, **Service**, and **Deployment** is. It does **not** know what an **AlloyDBCluster** or a **PostgreSQLBackup** is.

Kubernetes Custom Controllers — Quick Notes

1. What is a Custom Controller?

- **Core Concept:** An active reconciliation loop in Kubernetes that continuously monitors the state of custom resources and works to align the **actual state** of the cluster with the **desired state**.
- **Key Components:**
  - **Custom Resource Definition (CRD):** Defines the API schema for a custom object (e.g., defining a VirtualService resource).
  - **Custom Controller:** The actual logic/code watching that custom resource, handling its creation, update, and deletion logic.
- **Common Real-World Examples:**
  - **Ingress Controllers** (e.g., NGINX, Traefik)
  - **Service Mesh Controllers** (e.g., Istio managing VirtualServices and Gateways)
  - **Cert-Manager** (automating TLS certificate procurement)

- **CRD (Custom Resource Definition):** Tell them: *"I understand that an Operator starts with a CRD, which extends the K8s API to understand a new object type, like 'AlloyDBCluster'."*
- **The Reconciliation Loop:** Tell them: *"The 'Brain' of an Operator is the reconciliation loop, which constantly ensures the live database matches the YAML configuration."*
- **Domain-Specific Logic:** Tell them: *"Standard K8s objects don't understand 'Database Failover.' The Operator is where we bake in that domain-specific knowledge, like checking Patroni API before promoted a follower to leader."*

How to Build a Custom Controller

1. **Define the CRD (Custom Resource Definition)**
   - Create an OpenAPI v3 YAML schema detailing your custom object's fields.
   - Apply it to the cluster (kubectl apply -f crd.yaml).
2. **Select a Development Framework**
   - **Golang:** Operator SDK or Kubebuilder (Industry Standard).
   - **Python:** Kopf.
   - **Java:** Java Operator SDK.
3. **Implement the Reconciliation Loop**
   - Write the business logic:
     `$Reconcile() : Actual\ State \rightarrow Desired\ State$`

- Watch events (Create, Update, Delete) on the target CRD.

4. **Set Up Security & Deploy**
   - Configure **RBAC (Role-Based Access Control)** permissions allowing the controller to read/write specific API resources.
   - Package the binary into a Docker container and run it as a Deployment inside the cluster.

Why "RollingUpdate" isn't enough for Databases

In a **Deployment** (Stateless), a Rolling Update is perfect because every pod is the same. But in a **Database**, a "dumb" Rolling Update can be dangerous for three reasons:

1. **The Leader/Follower Order:** In a StatefulSet, a Rolling Update typically starts with the highest index (e.g., db-2) and moves down. If db-0 is the **Leader**, and the update hits it, the database will crash unless a "Switchover" happens first.
2. **Schema Migrations:** If V2 of your app requires a new database table, but V1 is still running, your app might crash. A standard RollingUpdate doesn't know how to pause the rollout, run a SQL migration script, and then continue.
3. **Data Consistency:** If you are upgrading the database engine itself (e.g., PG16 to PG17), you often need to run specific "Upgrade Utilities." A StatefulSet will just swap the binary and try to start the pod; if the data files are in the old format, the database won't start.

This is why you have custom CRDs for "Update": Your project uses custom logic because you need to coordinate the **Control Plane** and **Data Plane**. The update must:

- Update the **Standby** nodes first.
- Tell the Cluster Manager to perform a **Switchover** (move the Leader to a safe node).
- Verify health, then update the **Leader**.

Project view:

1. **The Cluster Manager is the "Operator" (The Brain)**
   The **Cluster Manager RPM** is the true Operator in your project.
   - **Why?** Because it is the "living" process that receives your **Custom CRDs** (YAMLs) and makes the intelligent decisions.
   - In Kubernetes, the "Operator" is the code that listens to the API. Your Cluster Manager is doing exactly that: it provides an API endpoint, accepts a kind: DBCluster request, and knows the logic to talk to Patroni or HAProxy.
2. **Ansible is the "Provider" (The Hands)**
   Ansible is the tool the Operator uses to get the work done.
   - **Why?** Ansible is great at "Task Execution" (installing RPMs, starting services, moving files). But Ansible is usually "one-and-done."
   - In a Kubernetes Operator, the code often uses "Providers" or "Handlers" to execute commands. In your case, Ansible is the **Automation Engine** that the Cluster Manager triggers to ensure the RHEL 9 VMs are configured correctly.

Assigning Pods to Nodes Or Pool and cluster node binding: (k8s scheduler)

Labels and Selectors:

- In a Kubernetes **Deployment, labels must be written inside the** pod template(**spec.template.metadata.labels**). They are not written inside the container spec.

Labels and annotations are used for tagging metadata to objects to organize cluster resources.

Labels: Labels are key-value pairs that can connect identifying metadata with Kubernetes objects.

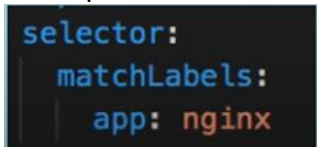
- They also helps to query using labels and performing bulk operations like deleting deployments with the specified labels.

Example : They connect deployments to pods.

  - Pods get the label through the template blueprint.
  - This label is matched by the selector.

Selectors:

- Selectors are to identify a set of resources based on their labels for management and operations.
- Spec block contains selector, It connects the deployment with the pod.



Types of Selectors:

- **Equality-Based Selectors:**

  Match labels that equal a specific value.
  Example: environment = production

- **Set-Based Selectors:**

  Match labels that belong to a set of values.
  Examples: environment in (production, staging), tier notin (frontend, backend)

Annotations: They are to attach metadata to objects in the form of key-value pairs.

- The metadata in an annotation can be small or large, structured or unstructured, and can include characters not permitted by labels.
- For example: registry address, build, release, or image information like timestamps, git branch.

Scheduling pods to nodes using labels:

1) nodeName
2) nodeSelector
3) Affinity & Anti-Affinity

1) **nodeName:** Directly assigning the value in the definition for the required argument.
    - The "nodename" field in a pod specification directly specifies the node (server) where you want the pod to run.

- **Why use it?**

  To explicitly place a pod on a specific node. This is useful for tasks like debugging or when you have a specific node that meets all the requirements for the pod.

    - It has highest precedence when it is mentioned.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  nodeName: node1  # This pod will run on the node named 'node1'
  containers:
  - name: my-container
    image: nginx
```

    - **Limitations** are when the selected node is insufficient to hold a new pod then it will be failed, for cloud based architecture node names are not static hence pod assignment will be difficult.

2) **nodeSelector:**

- The nodeSelector field in a pod specification allows you to specify a set of key-value pairs (labels) that a node must have for the pod to be scheduled onto it.
- Specifying the argument in the pod definition and give the respective labels in which nodes belong to.

- **Why use it?**

  To ensure that pods are scheduled only on nodes that meet specific criteria (e.g., nodes with certain hardware, in a specific location, or with particular capabilities).

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  nodeSelector:
    disktype: ssd  # This pod will run only on nodes with the label
'disktype=ssd'
  containers:
  - name: my-container
    image: nginx
```

Difference between Labels, Selectors vs Pod schedulers
Labels and **selectors** act like an internal indexing and tagging system to connect different Kubernetes objects together. **NodeSelectors**, on the other hand, act as hard scheduling rules that instruct the cluster exactly which worker nodes are allowed to run your pods.

Affinity & Anti-Affinity:

- Affinity and anti-affinity rules help control where pods are placed within a Kubernetes cluster by specifying preferences or constraints based on labels.
- It consists of four types:
  a. Node Affinity
  b. Node Anti-Affinity (using taints)
  c. Pod Affinity
  d. Pod Anti-Affinity

Node Affinity: Node affinity rules guide Kubernetes on which nodes (servers) are preferred or required for scheduling a pod based on node labels.

Node affinity is conceptually similar to nodeSelector, allowing you to constrain which nodes your Pod can be scheduled on based on node labels.

Difference between Node Selector vs Node Affinity:

Node Selector is a simple, strict label-matching tool. It requires an exact match to schedule a pod on a node.
Node Affinity is a more advanced, flexible system. It supports "soft" rules (preferences) and complex logical operators like `In`, `NotIn`, or `Exists`.

- **Why use it?**

To ensure that pods run on nodes with specific characteristics or resources.

- Attracting pods to be schedule on the basis of node labels.

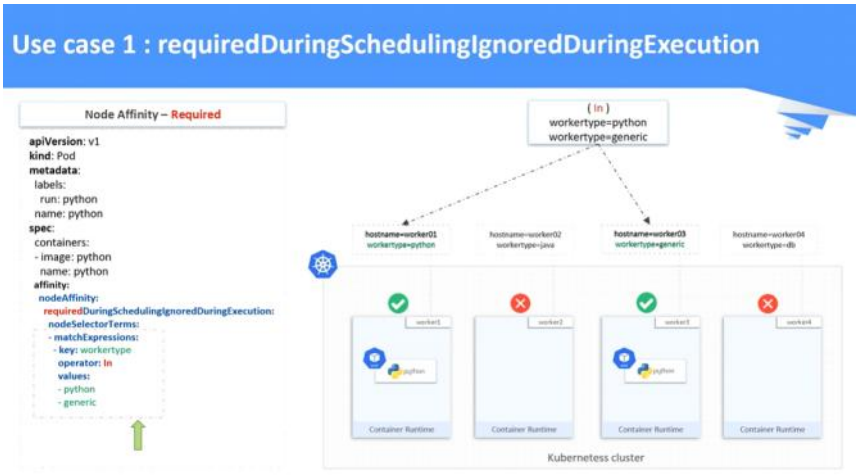
There are two types of node affinity :

1) requiredDuringSchedulingIgnoredDuringExecution:

- The pod can only be scheduled on nodes that meet these criteria.

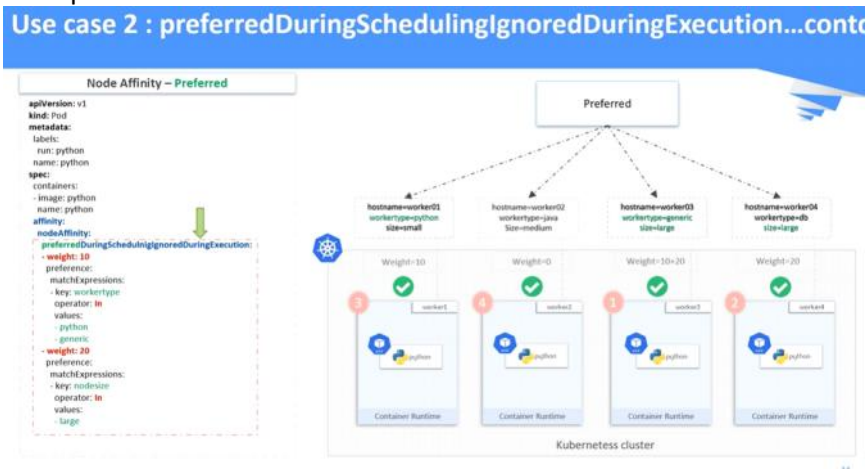
Ex: You have nodes labeled with region: us-west and want your pod to run only on those nodes.

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: region
          operator: In
          values:
      - us-west
```



2) preferredDuringSchedulingIgnoredDuringExecution:

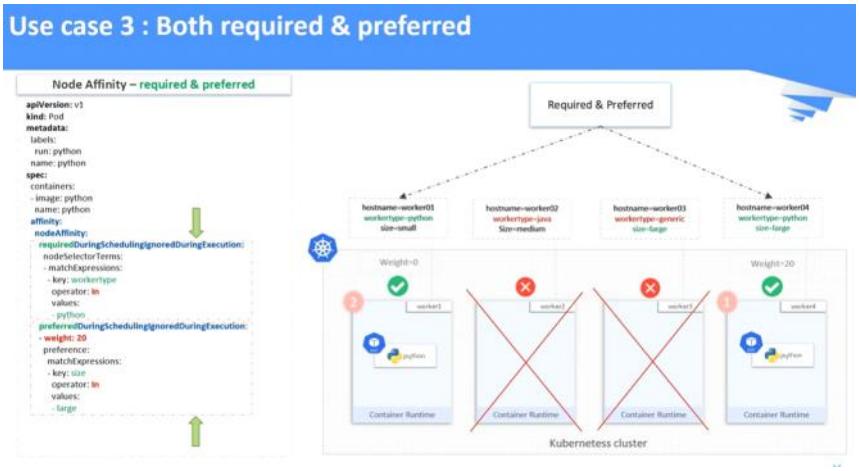
- Kubernetes tries to place the pod on nodes that meet these criteria but can place it elsewhere if necessary.
- The scheduler tries to find a node that meets the rule. If a matching node is not available, the scheduler still schedules the Pod.
- It is completely based on weightage preference to select a node to schedule a pod.



- **Operator** decides the values to be considered or not Ex:
  - Operator: in : The "In" operator is used to specify that a label key must have one of the specified values for the rule to match.
  - Operator: Notin : The "NotIn" operator is used to specify that a label key must not have any of the specified values for the rule to match.
  - Operator: Exists -> It checks whether the key is present or not in the nodes, It won't think about values.
- **ignoredDuringExcecution** means that if the node labels change after Kubernetes schedules the pod, the pod continuous to run.

We can use both types in a manifest file:

- Hence both rules need to apply for scheduling a pod.



Pod Affinity

Pod affinity rules guide Kubernetes to schedule a pod close to other specified pods, typically on the same node or within a particular topology.

- **Why use it?**

To enhance performance by keeping related pods together, such as those that communicate frequently.

Example:

You want new pods to be scheduled on the same nodes as existing pods with the label app: frontend.

```
affinity:
  podAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values:
          - frontend
    topologyKey: "kubernetes.io/hostname"
```

Pod Anti-Affinity

Pod anti-affinity rules guide Kubernetes to avoid placing a pod on the same node as certain other pods.

- **Why use it?**

To distribute pods across nodes for improved fault tolerance and reliability.

Example:

You want to ensure that new pods are not scheduled on the same nodes as existing pods with the label app: backend.

```yaml
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector:
        matchExpressions:
        - key: app
          operator: In
          values:
          - backend
    topologyKey: "kubernetes.io/hostname"
```

Feature	Labels + Node Affinity	Taints & Tolerations
Concept	**Attraction** (Pods reach out for Nodes)	**Repulsion** (Nodes push away Pods)
Default state	Nodes accept **any** Pod by default.	Nodes **reject** all Pods by default unless allowed.
Where set	Labels are set on Nodes; rules are set on Pods.	Taints are set on Nodes; tolerations are set on Pods.
Primary Goal	Directing specific Pods to specific Nodes.	Reserving Nodes or keeping unwanted Pods off Nodes.

Taints & Tolerations:

- Taints and tolerations work together to control which pods can be scheduled on which nodes. They provide a way to repel pods from certain nodes or to attract them to certain nodes only if they can tolerate the conditions on those nodes.

  Taints
  Taints are a way to mark a node so that no pods will be scheduled on it unless they explicitly tolerate the taint.

  Why use them?
  To reserve nodes for specific purposes or to prevent certain types of pods from being scheduled on nodes that have special requirements.

  How do they work?
- When you apply a taint to a node, it effectively says, "Pods without a matching toleration should stay away."
- Taint a node with key=value:effect

  Example:
  Taint a node with key=value:effect, for example, key=special, effect=NoSchedule.

  CMD: `kubectl taint nodes node1 key=special:NoSchedule`
  This command marks node1 so that no pods will be scheduled on it unless they have a toleration for the key=special taint.

  Tolerations
  Tolerations are a way to tell Kubernetes that a pod can be scheduled on a node with specific taints.

Why use them?
To allow certain pods to be scheduled on nodes that have taints, essentially making exceptions for specific pods.

How do they work?
A toleration on a pod allows it to tolerate a matching taint on a node.

Example:
A pod can tolerate the key=special taint.

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  tolerations:
  - key: "special"
    operator: "Exists"
    effect: "NoSchedule"
  containers:
  - name: my-container
    image: nginx
```

This configuration allows my-pod to be scheduled on nodes that have the key=special:NoSchedule taint.

How Taints and Tolerations Work Together

Applying Taints:
Nodes can be tainted to repel most pods.
Ex: Taint a node so that it doesn't accept general workload pods.

Adding Tolerations:
Specific pods can be given tolerations to allow them to bypass the taints and run on the tainted nodes.
Ex: Allow a special monitoring pod to run on a node that has been tainted to exclude regular workload pods.

Ex:

Taint a Node:

```
kubectl taint nodes node1 dedicated=experimental:NoSchedule
```

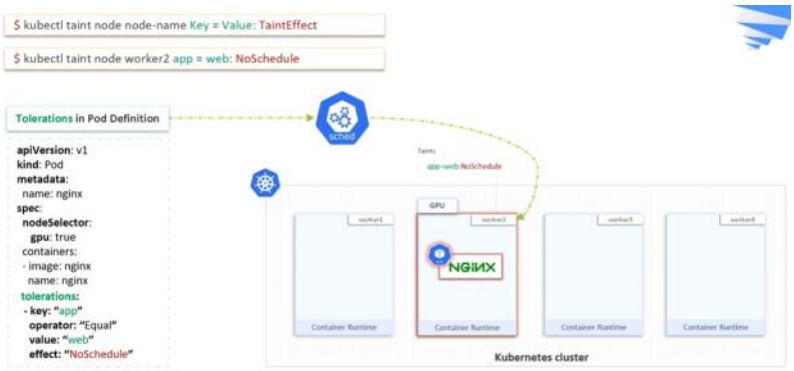
This taints node1 with dedicated=experimental:NoSchedule, meaning only pods with a toleration for this taint can be scheduled on node1.

Tolerate the Taint:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-experimental-pod
spec:
  tolerations:
  - key: "dedicated"
    operator: "Equal"
```

```
    value: "experimental"
    effect: "NoSchedule"
  containers:
  - name: my-container
  image: nginx
```

This pod will tolerate the dedicated=experimental:NoSchedule taint and can be scheduled on node1.



Ex: If a mosquito approaches a man with Taint(mosquito repellent spray on him) then the mosquito will not land on to that man. If a fly approaches a man Taint is tolerant for it as mosquito repellent spray can't do anything hence the fly will land on him.

- **Taint effects :**
  - **NoSchedule:** When taints are applied, existing pods remain on the node remain. New pods that do not match the taint are not scheduled onto that node.
  - **PreferNoSchedule:** When taints are applied, existing pods remain on the node remain. New pods that do not match the taint might be scheduled onto that node, but the scheduler tries not to.
  - **NoExecute:** When taints are applied, existing pods on the node that do not have a matching toleration are removed. New pods that do not match the taint cannot be scheduled onto that node.

- Selectors: They used a filters, the labels are matched by the selector
- It forward requests to pods with label of this value, a way of selecting labels.
- Specification part contain selectors
- These selectors are placed inside a service also here it identifies the label in it and matches the label with deployment likewise here they connecting services to deployments.

Pod affinity:

Inter-pod affinity and anti-affinity allow you to constrain which nodes your Pods can be scheduled on based on the labels of Pods already running on that node, instead of the node labels.

- Pod affinity/anti-affinity allows you to constrain which nodes your pod is eligible to be scheduled on based on the labels on other pods.
- Attracting pods to be schedule on the basis of pod labels on node.

Node Affinity vs Pod affinity:

Node Affinity ensures that pods are hosted on particular nodes. Pod Affinity ensures two pods to be co-located in a single node.

Taint and Toleration Services:

Taint : They are just opposite to Node affinity here, they allow a node to repel a set of pods.
Toleration: Tolerations are applied to pods, and allow (but do not require) the pods to schedule onto nodes with matching taints.

Ex: If a mosquito approaches a man with Taint(mosquito repellent spray on him) then the mosquito will not land on to that man. If a fly approaches a man Taint is tolerant for it as mosquito repellent spray can't do anything hence the fly will land on him.
-> Taints and tolerations work together to ensure that pods are not scheduled onto inappropriate nodes.

1. Liveness Probe: "Am I still alive (or frozen)?"

The Liveness Probe determines if the application has crashed or entered a "broken" state that only a restart can fix.

- **Behavior:** If this probe fails, Kubernetes **kills the container and restarts it** (based on the restartPolicy).
- **Use Case:** If your Java or Go process is running, but a thread has deadlocked and the API is no longer responding to requests, the Liveness probe will fail and force a fresh restart.
- **Caution:** Do not point this to a dependency (like a database). If the DB is down, you don't want all your web servers to restart in a loop—that's a "Death Spiral."

```yaml
spec:
  containers:
  - name: alloydb-app
    image: alloydb-omni:v16
    readinessProbe:
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 5  # Wait 5s before first check
      periodSeconds: 10       # Check every 10s
    livenessProbe:
      tcpSocket:
        port: 5432            # Check if Postgres port is responding
      initialDelaySeconds: 15 # Give the DB more time to start
      periodSeconds: 20
```

2. Readiness Probe: "Am I ready for traffic?"

The Readiness Probe tells Kubernetes when your application is fully initialized and ready to handle user requests.

- **Behavior:** If this probe fails, Kubernetes **removes the Pod's IP from the Service**. It does *not* kill the pod.
- **Use Case:** During startup, your AlloyDB Omni monitor agent might need 30 seconds to load metadata or connect to etcd. You don't want the Load Balancer (HAProxy) sending traffic to it until that connection is solid.
- **Key Benefit:** Ensures Zero-Downtime deployments. The old version only stops receiving traffic once the new version's readiness probe passes.

Deployment vs Liveness probe:

- **Deployment:** Restarts if the container **exits/ Crash/ Dies**.
- **Liveness Probe:** Restarts if the container **hangs/ unresponsive**.

Kubernetes will always restart a container if the process crashes.
If your application has a "Segmentation Fault" or the code hits exit(1) (means non zero code) , the container dies, and the Deployment/ReplicaSet restarts it immediately. You don't need a Liveness probe for that.

However, the Liveness probe is specifically for the **"Zombie State"**—situations where the process is **still running** but is **completely useless.**

1. The "Zombie" Scenario (Why Liveness is unique)

Imagine your AlloyDB monitor agent or a Java application:

- **The Process:** It's still running in the background (PID 123 is active).
- **The Problem:** The application has hit a **Deadlock**. The code is stuck in an infinite loop or waiting for a database connection that will never come. It is not "dead," but it is "frozen."
- **The Result:** Since the process hasn't "crashed," Kubernetes thinks everything is fine. Without a Liveness probe, that Pod will sit there forever, doing nothing and wasting resources.

The Liveness Probe is the only way to tell Kubernetes: *"The process is still running, but the application inside is 'brain-dead.' Please kill it and start over."*

2. Comparison: Process Crash vs. Application Deadlock

Scenario	Process Status	Will Deployment Restart it?	Role of Liveness Probe
App Crash (e.g. Out of Memory)	**Dead**	**Yes** (Automatically)	Not needed.
Deadlock / Freeze	**Alive**	**No** (K8s thinks it's okay)	**Required** to force a restart.
Infinite Loop	**Alive**	**No**	**Required** to catch the hang.

Application Deadlock:

- It is a specific type of logic failure where two or more threads (or processes) are stuck forever because each is waiting for the other to release a resource.
- Unlike a crash (where the app stops), in a deadlock, the app is **still running**, but it is **doing absolutely nothing**.

The "Deadly Embrace" (How it happens)

To have a deadlock, you typically need two threads and two resources (like two different tables in your AlloyDB database).

The Scenario:

a. **Thread A** locks **Table 1** and wants to update **Table 2**.
b. **Thread B** locks **Table 2** and wants to update **Table 1**.
c. **Thread A** waits for **Thread B** to release Table 2.
d. **Thread B** waits for **Thread A** to release Table 1.

Neither will ever let go. They are "frozen" in time.

Startup Probe: Indicates whether the application within the **container is started**.

1. The Problem: The "Restart Loop"

Without a Startup Probe, if you have a **Liveness Probe** configured to check every 10 seconds, but your application takes 60 seconds to boot:

a. K8s starts the container.
b. Liveness probe checks at 10 seconds. App isn't ready. **Fail.**

c. Liveness probe checks at 20 seconds. App still booting. **Fail.**
d. K8s thinks the app is "dead" and **restarts** it.

Result: The app never finishes booting because it's constantly being killed.

```
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
  periodSeconds: 10   # Very strict once running
startupProbe:
  httpGet:
    path: /healthz
    port: 8080
  failureThreshold: 30 # Allow 30 failures...
  periodSeconds: 10    # ...checked every 10s = 300 seconds (5 mins) total
grace period
```

2. The Solution: How Startup Probes Work

The Startup Probe acts as a "Gatekeeper."

- When a container starts, **only the Startup Probe runs.**
- The Liveness and Readiness probes are **disabled** until the Startup Probe succeeds.
- Once the Startup Probe passes once, it turns itself off forever, and the other probes take over.

Init Container:

Specialized containers that run before app containers in a Pod.

- Init containers can contain **utilities or setup scripts** not present in an app image.
- Can have one or more Init containers.
- Init containers always run to completion
- Each Init containers must complete successfully before the next one starts.
- After completion of all Init containers app containers will starts.

Difference from regular Containers:

- Init containers support all the fields and features of app containers, including resource limits, volumes, and security settings.
- Init containers do not support lifecycle, livenessProbe, readinessProbe, or startupProbe because they must run to completion before the Pod can be ready.

1. Pause Container

- **Purpose**: Acts as the infrastructure sandbox for the Pod.
- **Role**: It is the first container to start. It holds the shared Linux namespaces (Network, IPC, UTS, and optionally PID) and the Pod's IP address.
- **Lifecycle**: Runs continuously until the entire Pod is terminated.

2. Init Containers

- **Purpose**: Performs initialization tasks before app containers start.
  - Init containers can contain **utilities or setup scripts** not present in an app image.
- **Role**: Runs setup scripts, populates databases, or waits for external dependencies to become ready.
- **Lifecycle**: Executes sequentially to completion. If one fails, the Pod restarts (depending on restartPolicy).

3. App (Standard) Containers

- **Purpose**: Runs the core workload or business logic.
- **Role**: Executes the main application code (e.g., a web server or database instance).
- **Lifecycle**: Runs in parallel after all Init containers finish. They run continuously until the application stops.

4. Sidecar Containers

- **Purpose**: Enhances or extends the main application container.
- **Role**: Handles peripheral tasks like log shipping, service mesh routing, or metric collection.
- **Lifecycle**: Defined as Init containers with `restartPolicy: Always`. They start before app containers but run continuously alongside them.

5. Ephemeral Containers

- **Purpose**: Provides a temporary environment for live troubleshooting.
- **Role**: Attached dynamically to a running Pod using `kubectl debug` to inspect distroless or crashed containers.
- **Lifecycle**: Cannot be defined in the initial Pod spec. They are added on-demand and cannot be restarted.

RunAsUser: To specify security settings for a Pod, include the securityContext field in the Pod specification.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: security-context-demo
spec:
  securityContext:
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
  volumes:
  - name: sec-ctx-vol
    emptyDir: {}
  containers:
  - name: sec-ctx-demo
    image: busybox:1.28
    command: [ "sh", "-c", "sleep 1h" ]
    volumeMounts:
    - name: sec-ctx-vol
      mountPath: /data/demo
    securityContext:
      allowPrivilegeEscalation: false
```

Resource Management for Pods and Containers

Resource Request: The Kube-scheduler uses this information to decide which node to place the Pod on. Resources like CPU and memory.

Resource Limit: The Kubelet enforces those limits so that the running container is not allowed to use more of that resource than the limit you set.

- If the node where a **Pod** is running has enough of a resource available, it's possible (and **allowed**) for a container to use **more resource** than its request for that resource specifies. However, a **container** is **not allowed** to use more than its resource limit.
- *A Pod resource request/limit is the sum of the resource requests/limits of that type for each container in the Pod.*

Resource Quotas:

Provides constraints that limit aggregate resource consumption per namespace.

- When several users or teams share a cluster with a fixed number of nodes, there is a concern that one team could use more than its fair share of resources.

Kubernetes scopes and resources:

Scope	Resource Names
Cluster	Node, Namespace, PersistentVolume, StorageClass, PriorityClass, RuntimeClass, ClusterRole, ClusterRoleBinding, CustomResourceDefinition
Namespace	Deployment, StatefulSet, DaemonSet, Service, Ingress, PersistentVolumeClaim, ConfigMap, Secret, Role, RoleBinding, ResourceQuota, NetworkPolicy
Node	Node Lease, Image, Static Pod
Pod	Container, Init Container, Ephemeral Container, Volume (e.g., emptyDir, hostPath)

1.

Troubleshooting:

2. The "Troubleshooting" Scenario (The Hard One)

"I have a Headless Service, but when I run `nslookup`, I get an empty response. What could be wrong?"

The Answer:

- 1. **Selectors:** Check if the Service's `selector` actually matches the labels on your Pods.
- 2. **Pod Readiness:** By default, Kubernetes only puts "Ready" Pods in the DNS. If your Pods are failing their health checks, they won't show up in the Headless DNS list.
- 3. **publishNotReadyAddresses:** If you need the IPs even before the Pods are ready (common for initial database bootstrapping), you need to set `publishNotReadyAddresses: true` in the Service spec.

How do you handle your Kubernetes cluster security:

How do you handle your kubernetes cluster security?

There are many things that you can do, some of them are:

- By default, POD can communicate with any other POD, we can setup network policies to limit this communication between the PODs.
- RBAC (Role based access control)
- Use namespaces for multi tenancy
- Set the admission control policies to avoid running the privileged containers.
- Turn on audit logging.

How to achieve High Availability in Kubernetes

1. High Availability for the Control Plane

The Control Plane is the "brain" of the cluster. To make it HA, you must ensure there is no single point of failure.

- **Stacked Control Plane Nodes:** Deploy at least **three** control plane nodes. This ensures that if one node fails, the cluster maintains a **quorum** (majority) to make decisions.
- **Load Balancing the API Server:** Place an external or internal load balancer (like HAProxy or an Infrastructure Load Balancer) in front of the kube-apiserver on all three nodes. This provides a single virtual IP for worker nodes and admins to connect to.
- **etcd Cluster:** As the source of truth for the cluster, etcd must be distributed across the three nodes. You should mention your experience managing **etcd configuration and split-brain prevention** to show you understand the risks of data inconsistency.

2. High Availability for Worker Nodes (Data Plane)

This ensures your actual applications keep running even if hardware fails.

- **Multi-Zone Deployment:** Distribute worker nodes across different **Availability Zones (AZs)**. If an entire data center loses power, the nodes in the other zones remain active.
- **Infrastructure Scaling:** Use a **Cluster Autoscaler** to automatically add new VM nodes if the current ones become overwhelmed or fail.

3. High Availability at the Application Level

Even a perfect cluster won't help if your app is poorly configured.

- **ReplicaSets:** Always run at least **two or three replicas** of your pods.
- **Pod Anti-Affinity:** Use "Soft" or "Hard" Anti-Affinity rules to ensure replicas of the same app are **never** scheduled on the same physical node.

Summary Checklist for the Interview

Layer	HA Component	Goal
Control Plane	3+ Masters & Load Balancer	Redundancy for the K8s API.
Storage	Distributed etcd	Consistency and Quorum.
Infrastructure	Multi-Zone Nodes	Survival against Data Center failure.
App Logic	Anti-Affinity & PDBs	Ensuring replicas stay separated and

		available.
Connectivity	Ingress Controllers	HA entry points for external traffic.

Here is a clean, structured set of study notes you can copy directly:

What is Pod Disruption Budget:

Pod Disruption Budget (PDB) is a Kubernetes policy that **limits the number of Pods of a replicated application that can be down simultaneously** during planned maintenance. It acts as a safety net ensuring high availability for your services.

Voluntary vs. Involuntary Disruptions

A PDB protects your applications by targeting specific types of cluster actions:

- **Voluntary Disruptions (Protected):** Actions initiated by human administrators or automation tools. Examples include draining a node for an upgrade, cluster autoscaling, or application rollouts. The Kubernetes Eviction API checks the PDB before executing these actions and will block them if they breach your budget.
- **Involuntary Disruptions (Not Protected):** Unavoidable hardware or software failures. Examples include a physical node crashing, a kernel panic, or a network partition. PDBs cannot stop these real-world failures.

Core Configuration Parameters

You define a PDB using a label selector to target specific Pods, along with **one** of two metrics (either as an absolute integer or a percentage)

- **minAvailable:** The minimum number or percentage of Pods that **must remain active**. For instance, a quorum-based database with 3 replicas might set minAvailable: 2 to prevent losing its consensus state.
- **maxUnavailable:** The maximum number or percentage of Pods that **can be taken down** at one time. For a 10-replica stateless web service, setting maxUnavailable: 20% guarantees that at least 8 Pods are always routing traffic during

Imp: How would you handle environment isolation in k8 cluster

In Kubernetes, environment isolation (separating Dev, QA, and Prod) is typically handled using a layered approach. Since you are moving from a VM-based architecture where isolation is physical (different VMs/Projects), it is important to understand how Kubernetes does this

logically within the same cluster.

1. Primary Method: Namespaces

The most fundamental way to isolate environments is through **Namespaces**.

- **Logic:** You create a namespace for each environment (e.g., db-dev, db-qa, db-prod).
- **Isolation:** This provides a scope for names. You can have a service named alloydb in both dev and prod without them clashing.
- **RBAC:** You can restrict your team's access so they have "Admin" rights in dev but only "View" rights in prod.

2. Resource Quotas and Limits

To prevent a "Dev" application from accidentally consuming all the CPU and Memory of the cluster and crashing "Prod," you apply **Resource Quotas**.

- **Quotas:** Set a hard limit on the total resources a namespace can use (e.g., db-dev is capped at 10 CPUs).
- **LimitRanges:** Enforce default CPU/Memory requests and limits for every pod created in that namespace.

3. Network Isolation (Network Policies)

By default, all pods in a cluster can talk to each other. To achieve true isolation, you must implement **Network Policies**.

- **Rule:** Create a policy that denies all cross-namespace traffic.
- **Exception:** Explicitly allow only what is necessary (e.g., allow monitoring namespace to scrape metrics from db-prod, but block db-dev from talking to the db-prod database).

4. Node Isolation (Node Affinity/Taints)

For high-security or high-performance environments (like your **AlloyDB** production workloads), you can isolate the hardware itself.

- **Dedicated Nodes:** Use **Taints and Tolerations** to ensure that only Production pods can run on "Production-labeled" worker nodes. This prevents a buggy Dev app from causing "noisy neighbor" issues on the same physical VM as your Prod database.

Follow-up Question for your Colleague:

> "If we use **Namespaces** to isolate our Dev and Prod environments within the same cluster, how do we handle the **Secrets** (like database passwords)? Are Secrets shared across the cluster, or are they isolated to the namespace, and what happens if a developer with 'Read' access in Dev tries to access a Secret in Prod?"

The intended answer you're looking for:

- Secrets are **Namespaced** resources. They are strictly isolated.
- A developer's **RBAC** role is also namespaced. Even if they can "get secrets" in the dev namespace, the Kubernetes API will reject their request for the prod namespace unless they have a specific RoleBinding there.